

Introduzione

Abstract

This is a collection of personal notes for the course in *Introduction to Artificial Intelligence*, taken during the third year of my Bachelor's degree in *Artificial Intelligence and Data Analytics*, at the University of Trieste.

Some of the notes are direct transcripts from lecture notes taken during class, or are inspired by external materials (e.g. lecture notes made by other students, official textbook, official course notes for the CS 188 course at University of Berkeley)

The notes will mostly deal with the theoretical aspects with the course, albeit with (minimal) amount of exercises.

The following exercise is proposed to the reader, as usual:

Esercizio 1.

Find errors, mistakes or typos in the text, or missing content.

This page intentionally left blank.

Introduction to Artificial Intelligence

Introduction to AI

X

Introduction to Artificial Intelligence: Meaning of "intelligence", intelligence as mimicking human thinking or as rationality. Rational agents: utility functions, "maximize your expected utility". The brain: difference from software, similar aspects.

X

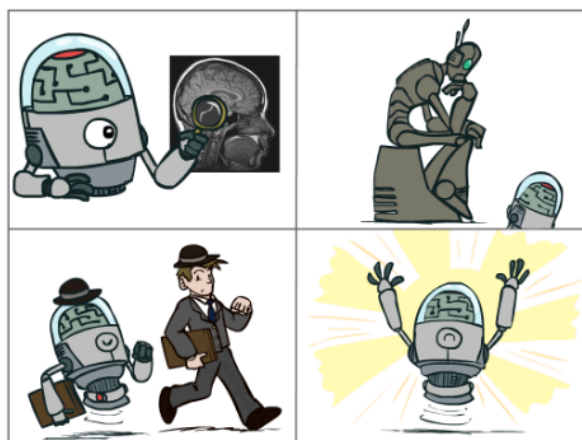
1. Meaning of Intelligence

For thousands of years, we have tried to understand how we *think* and *act*. Before asking what is *artificial intelligence*, one should actually understand what is meant by "*intelligence*":

- One can think of "*intelligence*" as the *observational intelligence* and the *internal processes* done during the reasoning
- Or can something "*intelligent*" just mimic the *human thinking process* as some sort of "*black box*"?

Clearly, we can see that there is no unique definition of *intelligence*. Still, we can define *artificial intelligence* as the process of creating "*intelligent agents*", which can *think* and *act like* either rationally or like human beings.

Think like people



Think rationally

Act like people

Act rationally

X

2. Rational Agents

A *rational agent* is an *agent* (something that *acts*, "*agere*") that acts in such a way that it is considered the "*best action*", in the sense that it achieves the best (or the best expected) outcome. Technically, with the term "*rational*" we also have other underlying meanings:

- The rational agent maximizes a *utility function*, which defines the goals to achieve
- We are only concerned with *the decisions made*, not *how* they are made

So in our context, rationality would represent the following slogan: "*Maximize Your Expected Utility*":

- *Maximize*: Find the best actions for each situation (state)
- *Your*: There can be more agents, in this case our rational agent will be "*selfish*". However, it is to be noted that in some cases a "*collective*" behaviour could lead to better results (e.g. games with shared resources); this topic is to be left for a more advanced course
- *Expected*: Usually the outcomes are not clearly defined, so they are usually estimated through probability distributions; in this case we will maximize our expectation
- *Utility*: The utility function will be used to model a situation or a problem to solve

Let us focus on the *utility function*. Note that it can be either *clearly defined* (e.g. Go, adversarial search, ...) or *not clearly defined* (e.g. Robotics); this aspect is very important, as the *utility function* will "*characterise*" the agent in a certain sense and thus it must account for everything.

X

3. The Brain

What about the *brain*? It is said that "*brains are to intelligence as wings are to flight*", in the sense that a brain can be thought as a form of intelligence, but it is not the only one. In fact, they are very good at making rational decisions, but not perfect. Moreover:

- Brains are *not modular* as software, meaning that they're hard to reverse engineer (or test on singular "components")
- Brains are still "*compositive*", meaning that some parts of the brains (the main areas) are focused on some aspects

However, one important aspect of the brain it is the fact that *memory* and *simulation* are keys to decision making; on machines, we will have *data* and *computation*.

History of AI

X

Brief overview on the history of Artificial Intelligence. Fields involved in AI. Precursor of AI: Aristotles. History of AI from the 18th century to modern day.

X

1. History of AI

Artificial Intelligence is a complex theme to tackle, and many fields are involved:

- Philosophy (reasoning, planning, learning, science, automation)
- Mathematics (first algorithm (Euclid), logic, probability, optimization)
- Neuroscience (neurons, adaptation)
- Economics (rationality, game theory for multi-agents modelling, operations research)
- Control theory (feedback)
- Psychology (learning, cognitive models, meaning of cognition)
- Linguistics (grammars, formal representation of meaning): this is an unexpected bridge between a *technical field* (AI) and a *humanistic field* (linguistics)

Now let us see some aspects on the history of AI.

Prehistory. We can already consider *Aristotle* (around 384-322 B.C.) to be a precursor of *Artificial Intelligence*, as he envisioned objects doing work "*by itself*":

"For if every instrument could accomplish its own work, obeying or anticipating the will of others [...] if, in like manner, the shuttle would weave and the plectrum touch the lyre without a hand to guide them, chief workmen would not want servants, nor masters slaves" (*Politics, Aristotle, 4th-century BC*)

Pre-Industrial Revolution:

- *1842*: Babbage's idea of the *Analytical Engine / Universal Machine* with subsequent Ada Lovelace's studies on it

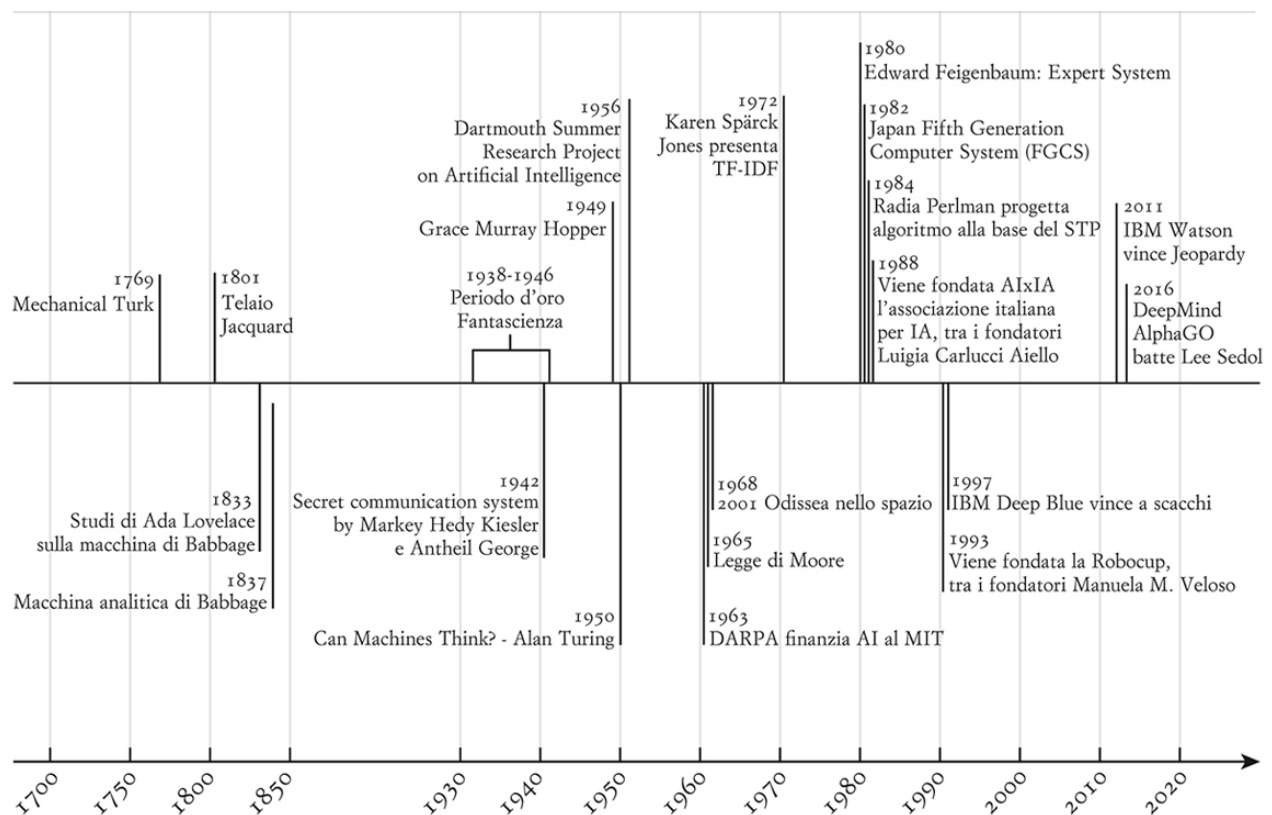
20th Century:

- *1943*: The first computational model of a *neuron*, the perceptron
- *1950*: Turing's test, question: "*Can outputs produced by machines be distinguished from the outputs made by the humans?*"
- *1956*: During a summer school workshop in Dartmouth, the computer scientist John McCarthy coined the term "*Artificial Intelligence*" for the first time
- *1956*: Proposal of project on Artificial Intelligence by John McCarthy and Claude Shannon

- **1956:** First "AI" program, *Logic Theorist*
- **50's:** Frank Rosenblatt's Perceptron
- **60's:** Logical reasoning (ELIZA, Robison's algorithm), SHRDLU and limitations of the perceptron
- **70's:** First "AI Winter", due to insufficiently strong hardware and state-space explosion; early development of knowledge-based systems
- **80's:** Backpropagation and Hopfield Networks: rise of Neural Networks. Connection machines, expert systems
- **90's:** Second *AI Winter*, scalability limits. *AI Spring*, rise of Machine Learning, Kasparov defeats Deep Blue at chess; one year later, Kasparov gets defeated by Deep Blue at chess

Today (2000's):

- Many subfields are getting merged
- AI appears everywhere
- AlphaGo defeats Lee Sedol at Go (2016)



Intelligent Agents

X

Notions on intelligent agents: definition and general diagram. High level description of agents: PEAS principle. Properties of task environments, consequences. Definition of reflex agents, model-based reflex agents, goal-based agents and utility-based agents.

X

0. Voci correlate

- Introduction to Artificial Intelligence

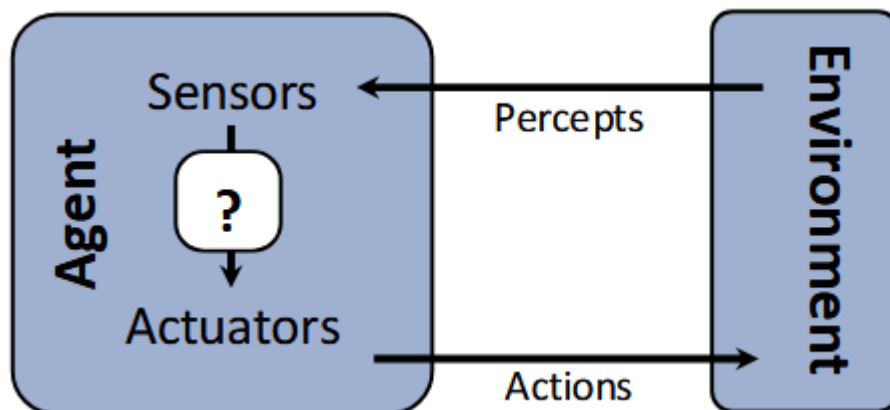
1. General Scheme of Intelligent Agents

#Definizione

Definizione 2 (intelligent agent).

An *intelligent agent* is an *entity* that *perceives* and *acts*, in such a way that its utility function is maximized. The actions are selected from the characteristics of the *sensors*, *actuators* and *environment*.

The diagram below shows the general scheme of an intelligent agent:



The boxed ? in the diagram is the *"technique"* of the agent and it characterises their decision-making process; it is the most crucial part of the intelligent agent, and we will see that certain *techniques* are *suitable* to *certain problem types*.

X

2. High-Level Description of Agents

To give a *high-level description of agents*, we will follow the so-called *"PEAS"* principle.

Lemma 3 (PEAS principle).

To design an *intelligent agent* in a high-level, one must describe the following components of the problem:

P: Performance Measure

E: Environment

A: Actuators

S: Sensors

Let us focus on the "*E*" part, that is the environment. We see that they can have the following properties:

- *Observability*: An environment can be either *fully* or *partially observable*
- *Multiplicity*: One or more agents might be involved
- *Transitions*: An environment may be *deterministic*, *non-deterministic* or *stochastic*
- *Static or Dynamic*: Whether the environment changes which each action taken by the agent(s)
- *Discrete or Continuous*: The environment might be described by an either *discrete* (i.e. countable) number of states, or a *continuous* one.
- *Knowledge of Rules*: We might know some underlying rules on the environment, which may be integrated in the agent
- *Knowledge of the Performance Measure*: Whether a performance measure can be either clearly described or not.

#Osservazione

Osservazione 4 (difference between non-determinism and stochasticity).

An environment is *not deterministic* iff by taking actions I do not have any known information about the possible outcomes. On the contrary, it's *stochastic* iff it's not sure but I know an underlying probability distribution which determines the possible next states.

In other words, *stochasticity* assumes the presence of a *probability distribution*.

Note that the *environment properties* might determine the *design of an agent*. For instance:

- Partial Observability implies needing *memory*
- Stochasticity implies having to prepare for unexpected situations
- More agents implies needing to behave *randomly*
- Staticity implies having time to compute *rational decisions*; otherwise the agent has a limited amount of time

- Continuity implies having to *operate a controller*
- Unknown physics implies the need of *exploring* and *observing/interacting*

X

3. Agent Types

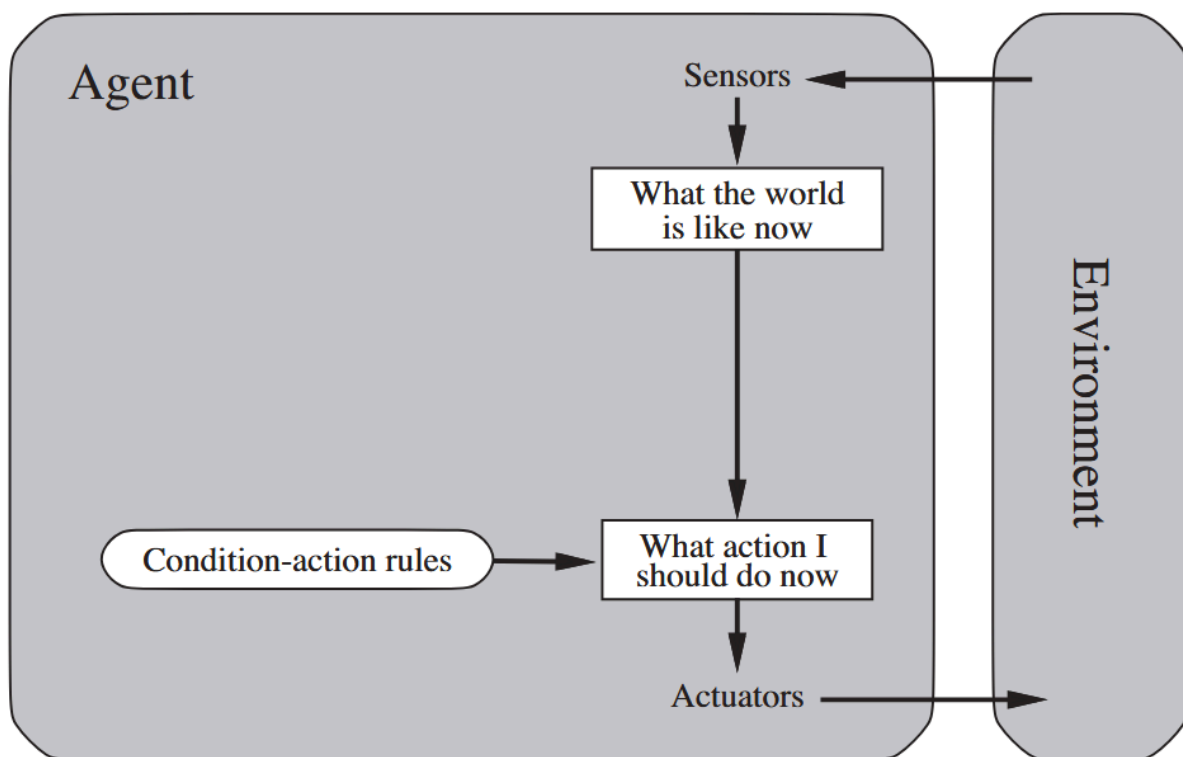
Let us see the main agent types.

3.1. Reflex Agents

#Definizione

Definizione 5 (reflex agents).

A *reflex agent* simply chooses the action based on the *current perception of the environment*. It never considers the *future consequences of their actions*



Of course this type of agent is the one to have *most limitations*, as it never considers the future states.

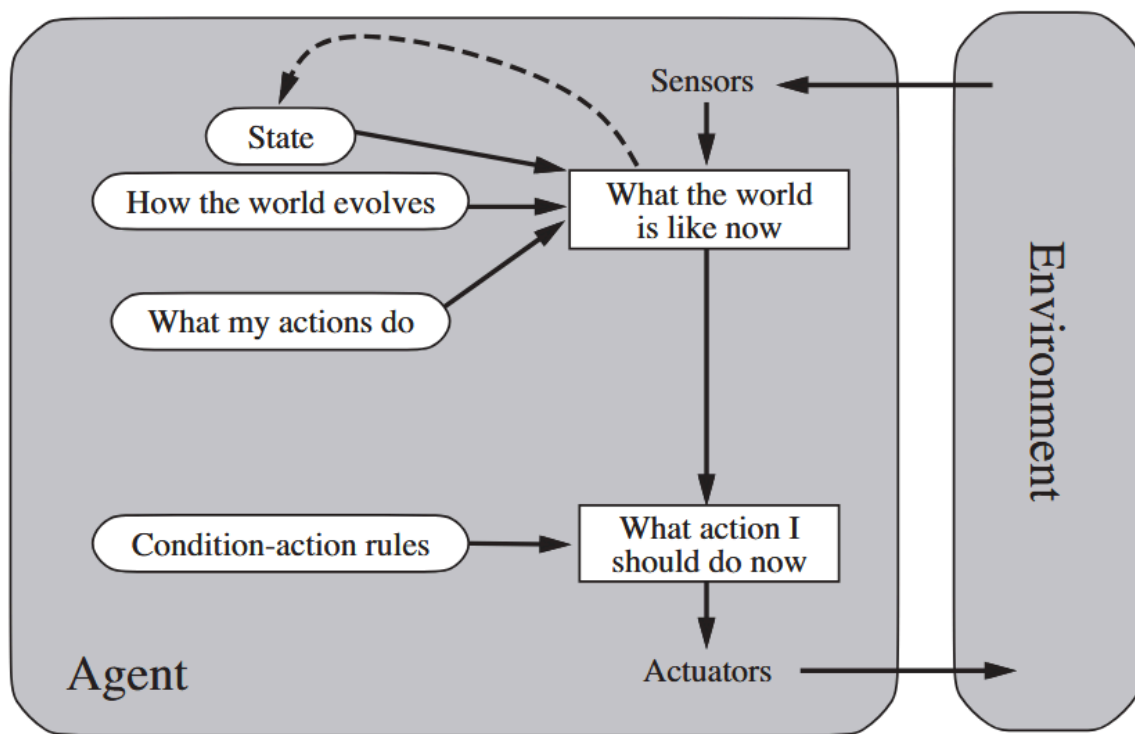
3.2. Model-Based Agents

#Definizione

Definizione 6 (model based agents).

A *model-based based reflex agent* contains a *more complex representation of the world* as well, so it might have memory on the states, the evolution of the world and the

consequences of their actions.

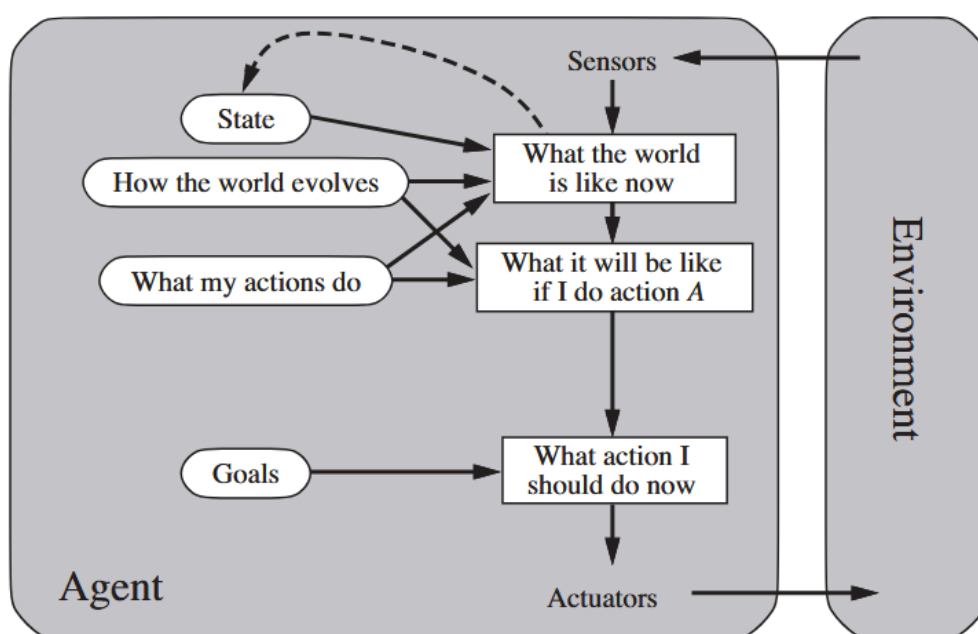


3.3. Goal-Based Agents

#Definizione

Definizione 7 (goal-based agent).

A *goal-based agent* works by *considering the consequences of their actions*, and it does by using a *model* which describes how the world evolves in response to a certain action. This is done by formulating a *goal*, some sort of *hypothesis* on "*how the world would be*"



Some limitations of their agents are contained in the step "*What it will be like if I do action A*", namely:

- Real-time hardware
- Implementations

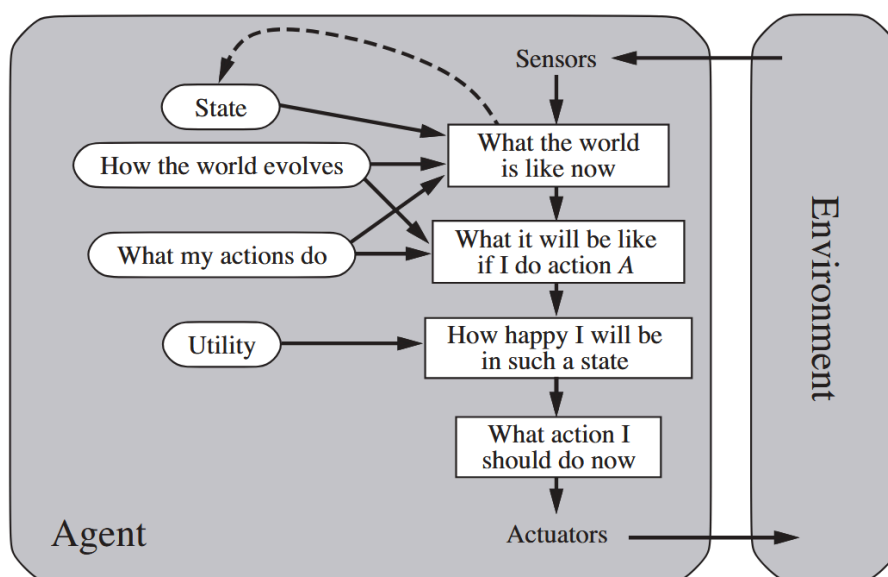
To improve these problems, we will see *heuristics*

3.4. Utility-Based Agents

#Definizione

Definizione 8 (utility-based agents).

An agent is *utility-based* if it's like the *goal-based agent*, with an extra step: it considers how it will be "*happy*" by factoring in the utility function.

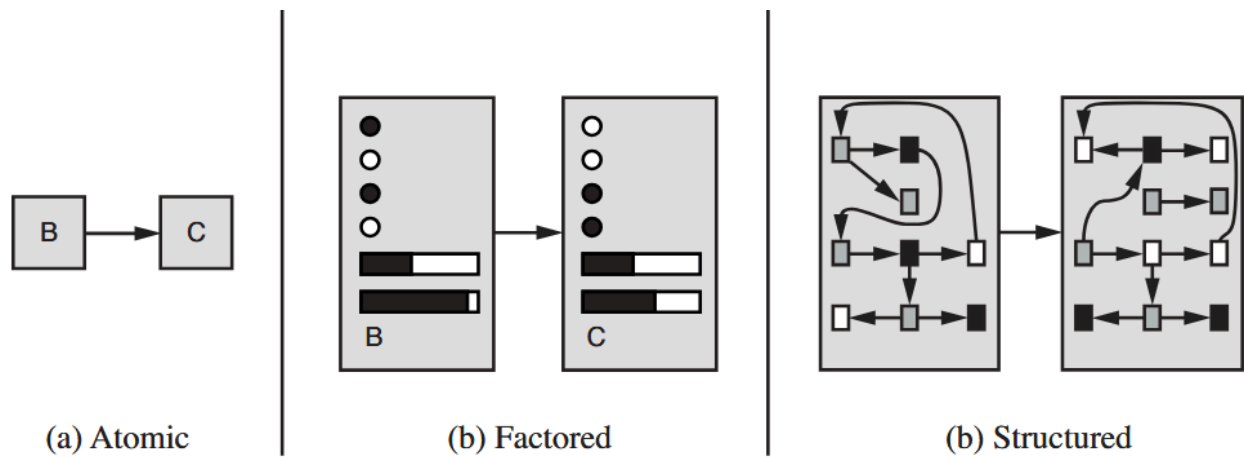


X

4. Agent Representations

The components of an agent's program interact in a complex way. To represent states and the transitions between them, we have three ways:

- *Atomic*: All the states are just nodes with no internal structure
 - Reflex agents deal with this well
- *Factored*: The states are *vectors* of *attribute values*
 - Model-based agents can deal with this
- *Structured*: A state includes *objects*, which might have attributes or relationships to other objects
 - In this case the agent must think on the actions



By also taking into account the *determinism* or *stochasticity* of the environment, we can classify problems in the following way:

- *Atomic and Deterministic* problems consist in *search problems*
- *Factored and Deterministic* problems consist in *logical problems* (propositional)
- *Structured and Deterministic* problems consist in *first-order logic problems* (we will not see this in depth)
- *Atomic and Stochastic* problems consist in exploring *Markov Decision Processes*

We will not see the rest of the cases.

Examples of Intelligent Agents

X

Examples of designing environments and agents: Pacman and vaccum cleaners.

X

0. Voci correlate

- Intelligent Agents

1. Example of Environments

Example. (the Pac-Man game)

Let us design, as a first example, the Pac-Man game as an environment. First of all, we must determine the state space $\mathcal{S}$. Let us assume we have a world with width w and length l .

- The amount of Pacman's possible positions on the grid are around $w \cdot l$
- Similarly, the two ghosts can be also placed in $w \cdot l$ positions.
- Regarding the *pellets' positions*, we have 2^{wl} combinations as each spot can "*contain a pellet or not*".

So, in total we have an environment with the state space of size $|\mathcal{S}| \simeq (wl)^3 \cdot 2^{wl}$.



X

2. Example of Agents

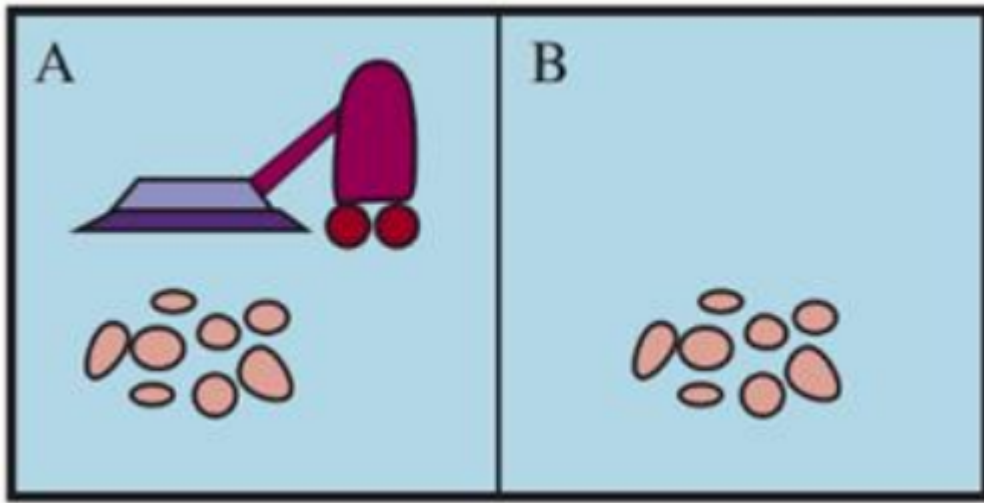
Example. (Vacuum Cleaners)

Let us design a *super simplified* design of a "*vacuum cleaner robot*". We let the environment consist in two spots, with each one having the possibility of being either clean or dirty.

Therefore $S = \{A, B\} \times \{\text{DIRTY}, \text{CLEAN}\}$. Let the agent perceive only the current tile it is on, and make it be able to either suck (cleaning the dirt) or move to the other tile. Our *reflex agent*, given a history of states-actions $\mathcal{D} = (s_0, a_0, s_1, a_1, \dots, s_t)$, will work like this:

$$\pi(\mathcal{D}) = \pi(s_t) = \begin{cases} \text{Suck}, s_t = (\bullet, \text{DIRTY}) \\ \text{Move to } (A, ?), s_t = (B, \text{CLEAN}) \\ \text{Move to } (B, ?), s_t = (A, \text{CLEAN}) \end{cases}$$

What's missing is the *performance measure*, or some sort of *numerical reward* to evaluate the agent. In this case the reflex agents might work well; however, it might fail in other cases. By the *reward hypothesis*, it should be sufficient to design a correctly-specified performance measure to have an agent that works well.



Example. (Vacuum Cleaners, part two)

So a *"better way"* to design a vacuum cleaner agent is to assume we *know the "geography" of the environment*, but the dirt distribution of each tile and the initial locations are unknown. Let the performance measure be calculated as follows:

$$\eta_t = \eta_{t-1} + \mathbb{1}(s_t = (\bullet, \text{CLEAN}))$$

Let η_t be calculated until $t = T = 1000$.

Q. Is the previously agent well-designed? The performance measure might seem to encourage the agent to *clean* the tiles, but it actually encourages the agent to *move* towards the clean tiles, and cleaning only if both are dirty. To improve it, one could consider adding a *"penalty"* for staying on dirty squares.

Example. (Pac Man)

Let us consider *Pac Man's* example, containing only pellets. We assume that Pacman is a reflex agent who can perceive the whole environment (so the pellets' positions), and must move in one of the four directions. The function for taking actions is defined as follows:

$$\pi(\mathcal{D}) = \pi(s_t) : \text{Move to the adjacent spot with a pellet}$$

However, it achieves the maximal performance measure only for simple cases, that is the path of the dots are fully connected. Therefore, this does not work for "*complex cases*"; to design a more robust agent, one should consider goal-based or utility-based agents.

Search Problems

Definition Search Problems

X

*General notions on search problems. Context of search problems, example of a search problem.
Formulating a search problem: state space, actions set, cost function and initial/goal states.
Example: pacman. Remark: differences between world state and search state.*

X

0. Voci correlate

- Intelligent Agents

1. Definition of a Search Problem

Suppose, in the context of "*search problems*", that agents are *goal-based* with atomic representation of the environment. Moreover, let the environment be episodic, single-agent, fully observable, deterministic, static, discrete and known.

#Definizione

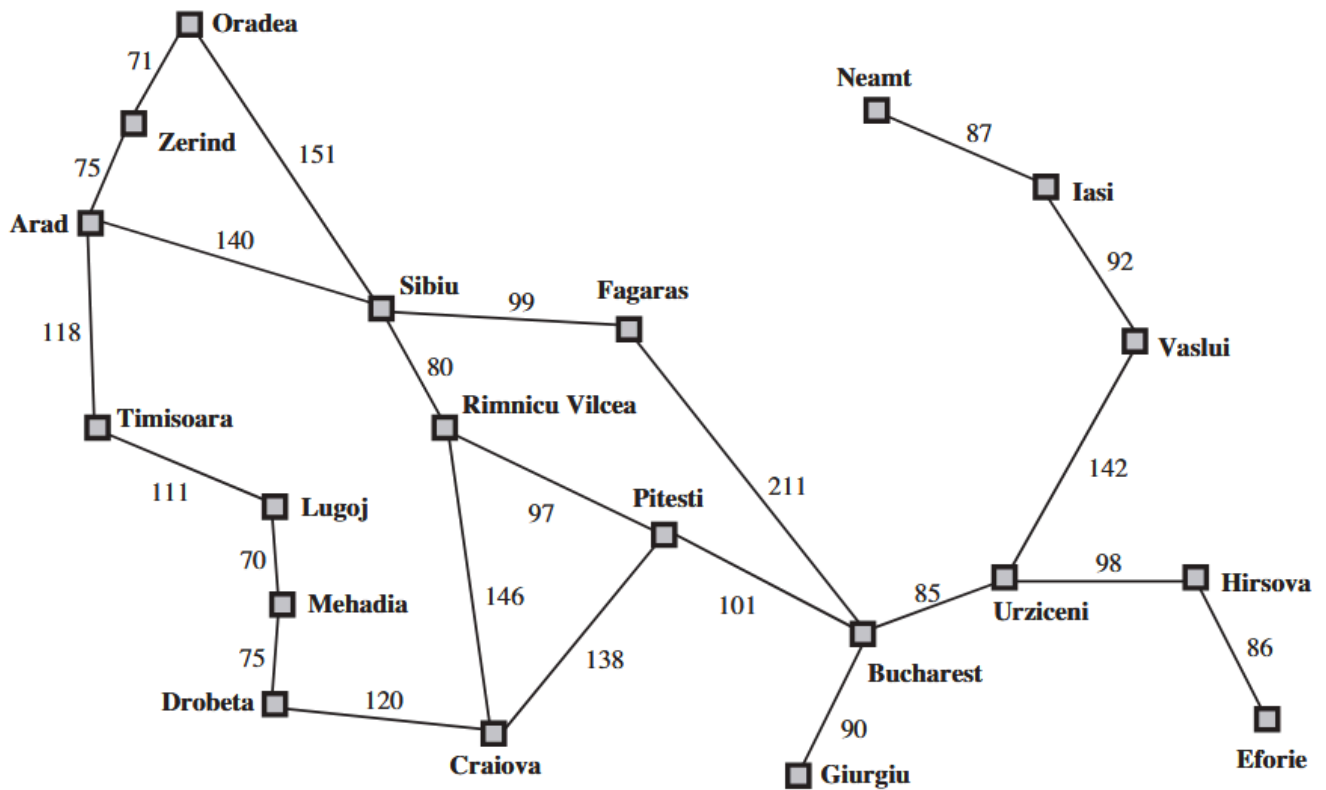
Definizione 9 (search problem).

A *search problem* is a *model* of the problem that consists of:

- A search state space $\mathcal{S}$
 - They can be *configurations* of the problem, which contains the necessary information for the agent
- An action set $\mathcal{A}$, moreover let $\text{actions}(s) \subseteq \mathcal{A}$ be the "*feasible moves*" on a certain state s .
 - Note that the actions which can be taken depends on the current state!
- A deterministic successor function which maps a state-action pair to a successive state. By convention, we denote it with $a(s) = s'$ where a is the action taken
- A cost function $c(s, a, s')$ where $s' = a(s)$
- An *initial state* and a *set of goal states*

We say that a *solution* is a path which transforms the goal state to a goal state. It is *optimal* if it has the lowest path cost among all solutions

Example. A simple example is *travelling* in a graph, with positive costs defined on archs:



#Osservazione

Osservazione 10 (difference between world state and search state).

Note that the *search state space* is not necessarily the same as the *world state space*. The world state space contains *all possible information* on the environments, whereas the *search state space* retains only the necessary information for resolving the problem.

So, in a certain sense, every search-state space differs from each other according to the goal that the agent has to accomplish.

Representation of Search States Space

X

Representation of search states space. State-space graphs, state-space trees. Remark: cycles in graphs implies infinite subtrees.

X

0. Voci correlate

- Definition of Search Problems
- Grafi

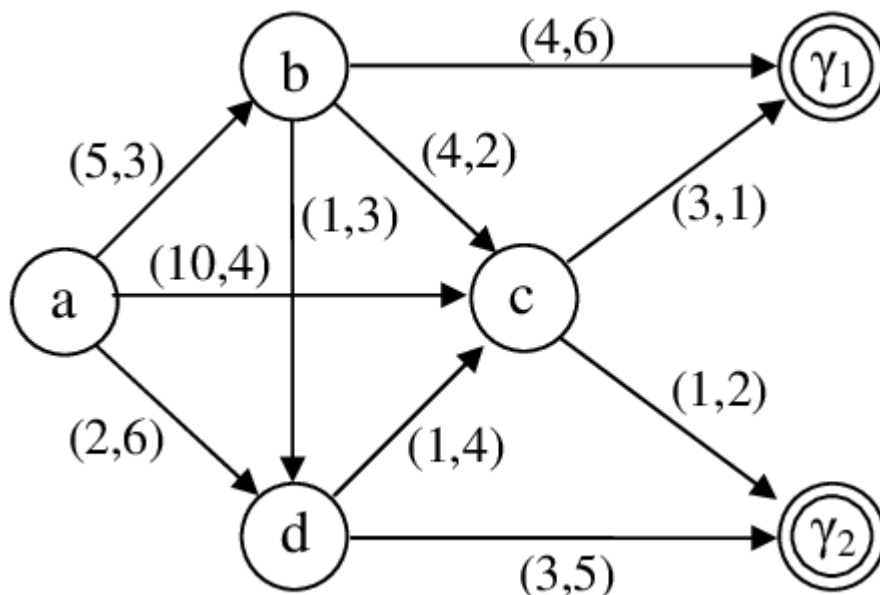
1. State-Space Graphs

Q. Suppose we have a *search problem* represented by a certain set of states $\mathcal{S}$, actions $\mathcal{A}$, successor function $a(s) = s'$ and cost function $c(s, a, s')$. How can we represent such a problem?

One straightforward and immediate way to do it is with the mathematical notion of *graphs*. That is, to construct a weighted directed graph $G = (V, E, \omega)$ where:

- $V = \mathcal{S}$
- $(s, s') \in E$ if and only if $\exists a : a(s) = s'$; if it's true, then assign the weight $\omega(s, s') = c(s, a, s')$

Moreover, we can keep track of the goal states in a separate set.



Note each state appears *only once*. This is a useful and intuitive idea, however it is rarely possible to build such graph in memory as state spaces tend to be big.

X

2. State-Space Trees

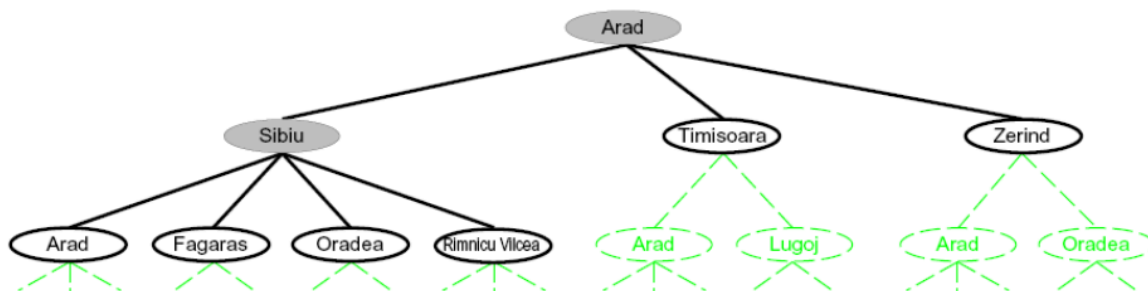
A better way to do it is through *state space trees*. It can be considered a some sort of *roll-out* of the *state space graph*, branching each node by "*what ifs*". The tree is constructed as follows:

- The root node starts from the start state
- Children corresponds to successors
- Nodes show states, but also *correspond to plans that achieve those states* (we just have to backtrack)
- It is built on *demand* by the agent

We observe the following property:

- If the *state-space graph contains loops*, then the *state-space tree is certainly infinitely deep*

From a practical point of view, we will use *spatate-space trees* and use *search algorithms on trees* to find a solution. The idea is mainly to *branch the tree nodes*, maintain a *fringe* of partial plans under construction and continue expanding nodes while expanding as few tree nodes as possible (for optimality).



PSEUDOCODE. (*General search tree algorithm*)

1. Let the problem and strategy be known
2. Initialize the search tree with the *initial state*
3. Loop infinitely:
 1. If there are no *successors* of the fringe then return *failure*
 2. Else:
 1. Choose a leaf node for expansion according to the *strategy* and remove it from the fringe
 2. If the leaf node contains a goal state, return *success*. Otherwise expand the node and add the resulting nodes to the search tree (in particular to the *fringe*)

Note that the *search strategy* is crucial.

Uninformed Search Algorithms

X

Uninformed search algorithms. Premise: simplified structure of search trees, estimation of amount of nodes to explore. Motivation to consider the time and size complexity, as well as soundness, completeness and optimality. Algorithms: DFS, BFS, DFS variants and UCS.

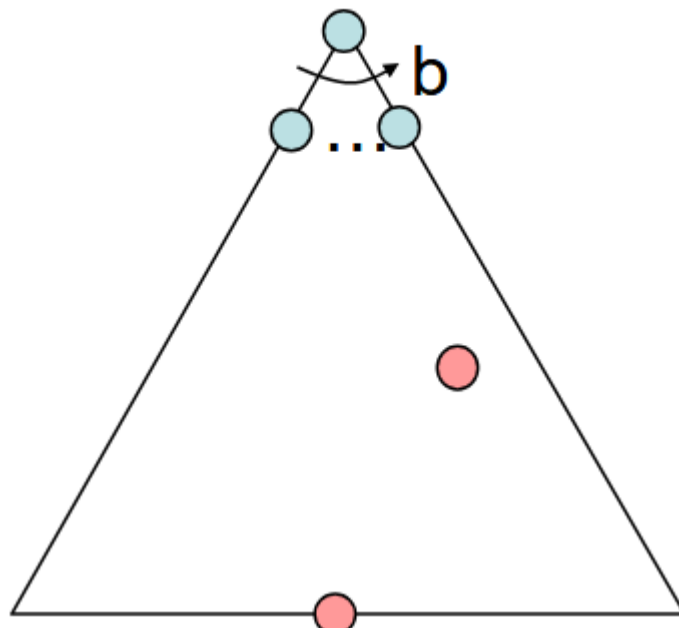
X

0. Voci correlate

- Definition of Search Problems
- Representation of Search States Space
- Ricerca in Profondità
- Ricerca in Ampiezza
- Algoritmo di Dijkstra

1. Premise

Note how *search trees* can be simplified as follows: let b the *branching factor of the tree*, which can be average of successors per node. Suppose we have defined a "*maximal amount*" of tiers which can be explored. Then, the *search "pyramid"* could contain the possible solutions.



Note how the amount of nodes can be estimated to $O(b^m)$, namely

$$1 + b + \dots + b^m = O(b^m)$$

This is why it will be important consider the *complexity* of *search algorithms*, as they can increase exponentially.

Given an *algorithm*, let us define its properties:

- *Soundness*: Can the algorithm return a "*correctly-defined*" solution?
- *Completeness*: Given there is a solution, can the algorithm guarantee to find one?
- *Optimality*: Is the solution given by the algorithm the (or one of the) optimal one(s)?
- *Time, Size Complexity*: How does the algorithm scale, with respect to the amount of steps or memory size required?

X

2. BFS and DFS

Idea. We can simply implement *fringes* as LIFO or FIFO data structures. In a sense, we will implement *depth-first* or *breadth-first* searches.

ALGORITHM. (DFS)

In DFS:

- The *fringe* is represented as a *LIFO* stack
- It expands the deepest nodes first as a consequence
- Tends to find the "*left-most*" solutions

Properties:

- Sound
- Incomplete
- Not optimal
- Time complexity: $O(b^m)$ if $m < +\infty$
- Size complexity: $O(bm)$, as we will explore at most m layers, then we multiply by b as it's the branching factor for "each layer"

Now let us move to the BFS

ALGORITHM. (BFS)

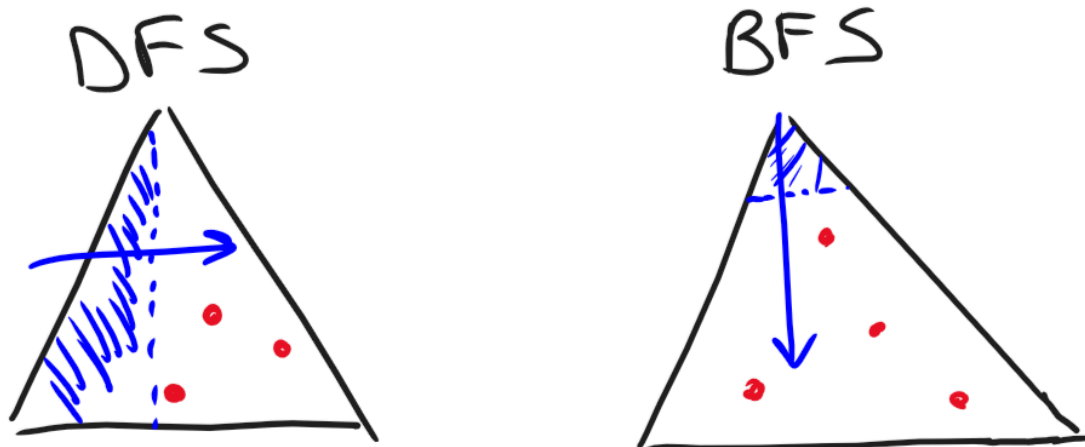
In BFS:

- The *fringe* is represented as a *FIFO* queue
- Thus it will explore the state space tree layer by layer

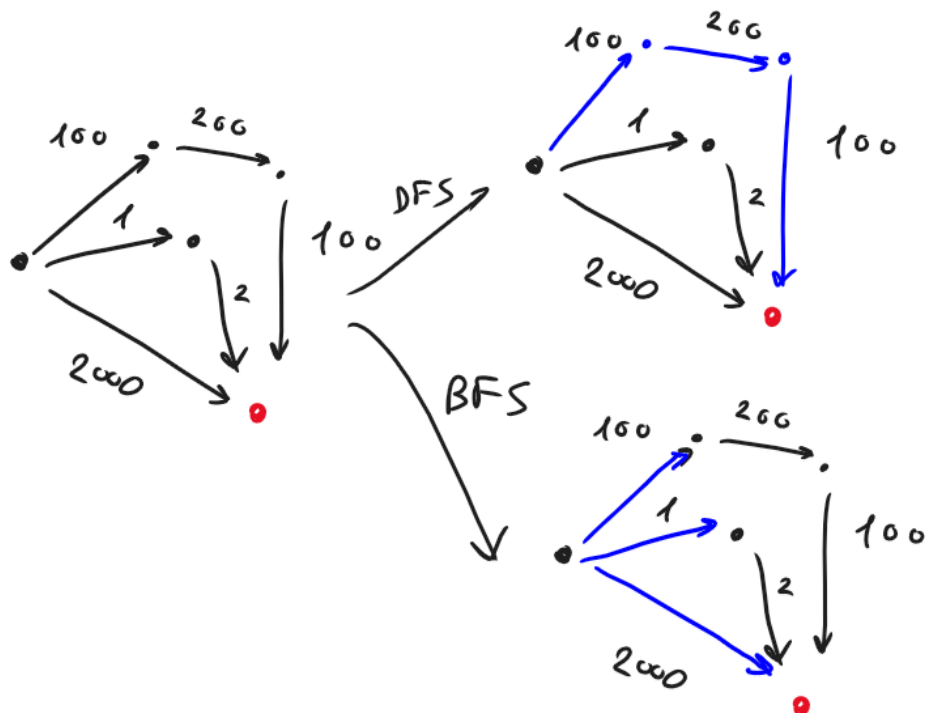
Properties:

- Sound
- Complete
- Not optimal
- Time complexity: $O(b^d)$, where d is the depth of the "*shallowest*" solution
- Space complexity: $O(b^d)$, as each level of the graph must be saved in order to generate the

next level



To discuss the *non-optimality* of the algorithms, we can simply devise simple counterexamples. Namely



Q. When can BFS outperform DFS? When can DFS outperform BFS?

Example: In the travelling salesman problem (TSP), DFS tends to outperform BFS as all of the solutions are on the "*same tier*". In a way, BFS won't be able to leverage the d factor as well as DFS does.

3. DFS Variants

DFS is an incomplete algorithm. How can we improve it and make it complete?

We have two variants of DFS:

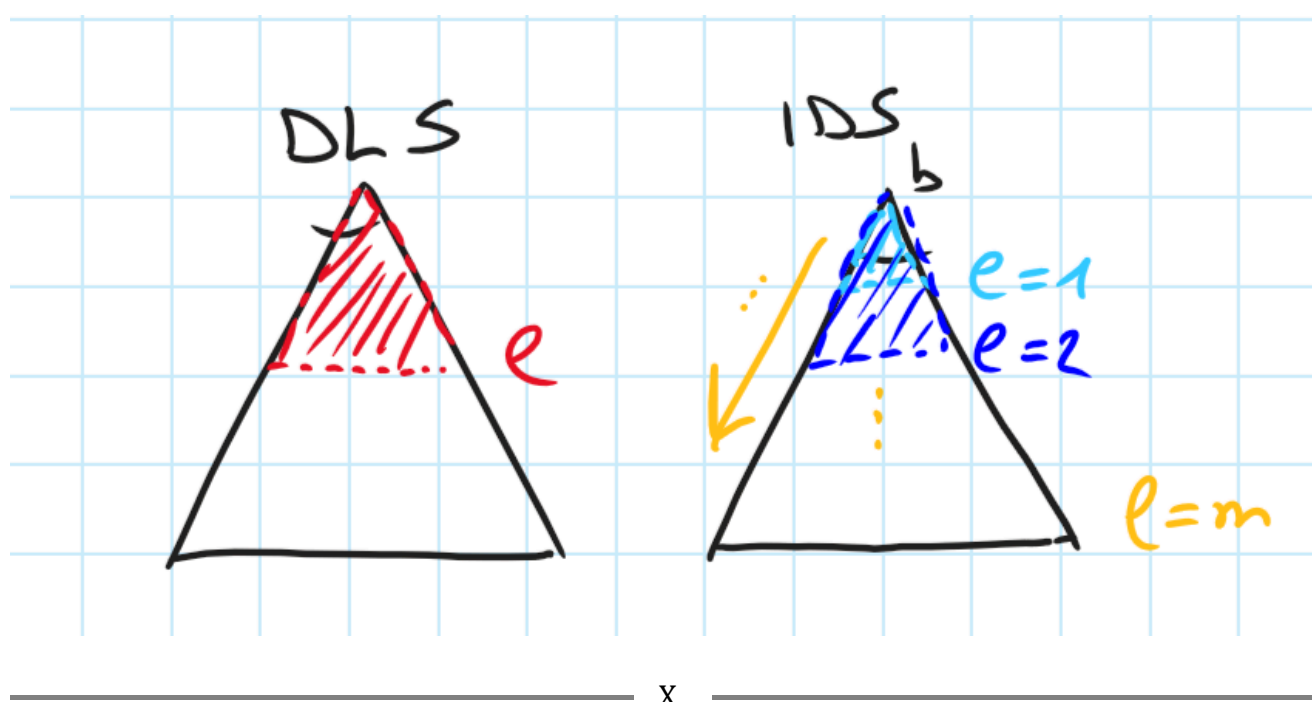
ALGORITHM. (*Depth-Limited DFS*)

The main idea is to truncate the search tree to the l -th tier. It can be useful when l is known in such a way that a solution can be found

ALGORITHM. (*Iterative Deepening DFS*)

In this variant we execute the depth-limited variant more times, increasing from 1 to the eventual amount m . In most cases, it finishes for low values of l . In a certain sense, this is effectively the same as BFS; however, this uses way less memory ($O(d)$).

Both of the algorithms are complete.



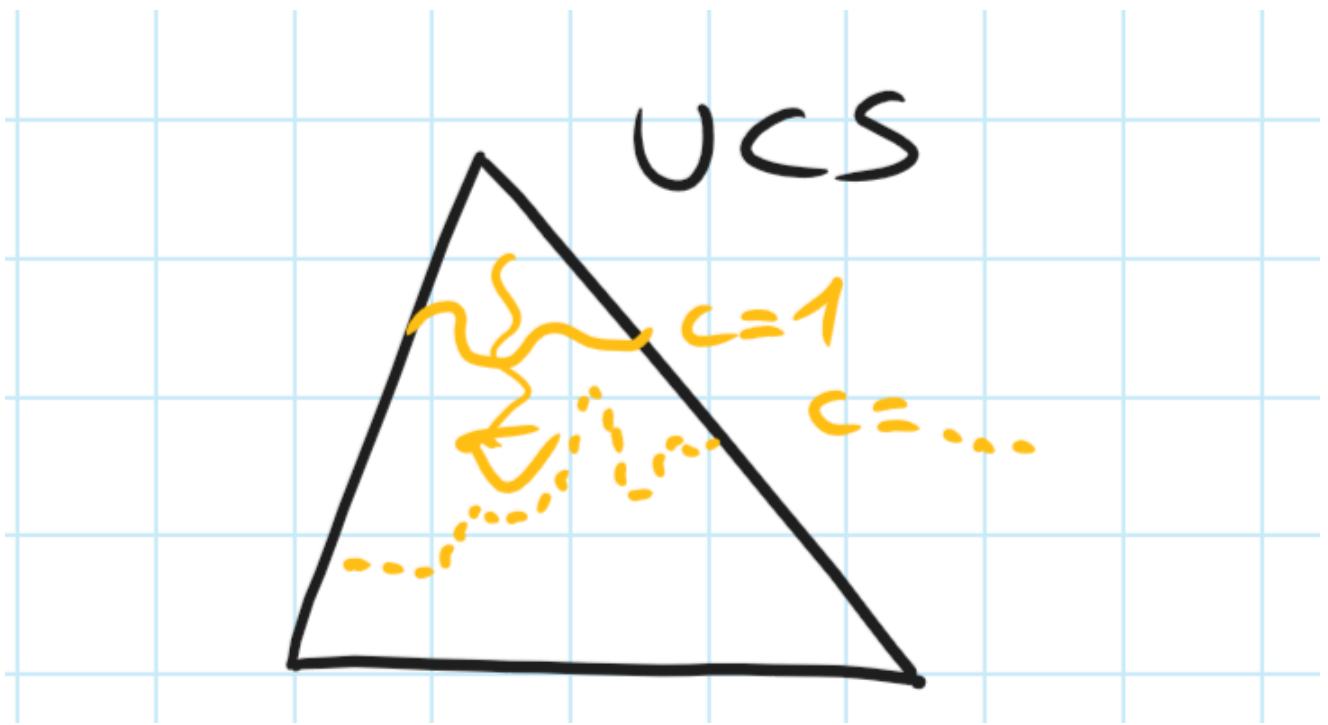
4. Unicast Cost Search (Dijkstra)

Lastly, we have an algorithm that is both *complete* and *optimal*. The algorithm is commonly known as Dijkstra's algorithm.

ALGORITHM. (*Unicast Cost Search*)

The main idea is to use a priority queue ordered on the *cost*, which is Dijkstra. In a sense, it draws a "*cost contour*" on the search tree and progresses on deepening the contours until the optimal solution is found. The algorithm has the following properties:

- Completeness and optimality
- Time complexity: let C^* be the solution cost and ϵ the least-cost constant, then UCS finds the solution in $O(b^{f+C^*/\epsilon})$ time. The space complexity is the same as all nodes need to be kept



The only "*problem*" with this algorithm is that it *explores every direction*, so it still doesn't leverage information about the goal whatsoever. The algorithms which do that are *informed search algorithms*.

Informed Search Algorithms

X

Informed search algorithms. Main idea of informed search: notion of heuristic. Ways to use heuristics: greedy search, A search. Properties on heuristics: admissibility and consistency. Theorem: A* search with an admissible heuristic is optimal. Ways to construct admissible heuristics. Comparing heuristics: dominance, maximum of admissible heuristics.*

X

0. Voci correlate

- Uninformed Search Algorithms

1. Idea of Informed Search

The main problem about *uninformed search algorithms* is that they expand all of the directions, being less efficient although surely optimal. On the other hand, the *informed search algorithms* have some "*ideas*" where to go, and it's relation to the goal state. We will formulate this idea with the notion of *heuristic functions*, i.e. a mapping from the state space to a real value:

$$h : \mathcal{S} \longrightarrow \mathbb{R}$$

In a way, h can be thought as an "*estimate of the distance from the goal state*".

However, we also have the weight costs $\omega(s, s')$; which one do we use, ω , h or a combination of them? Each informed algorithm has its own way to use h

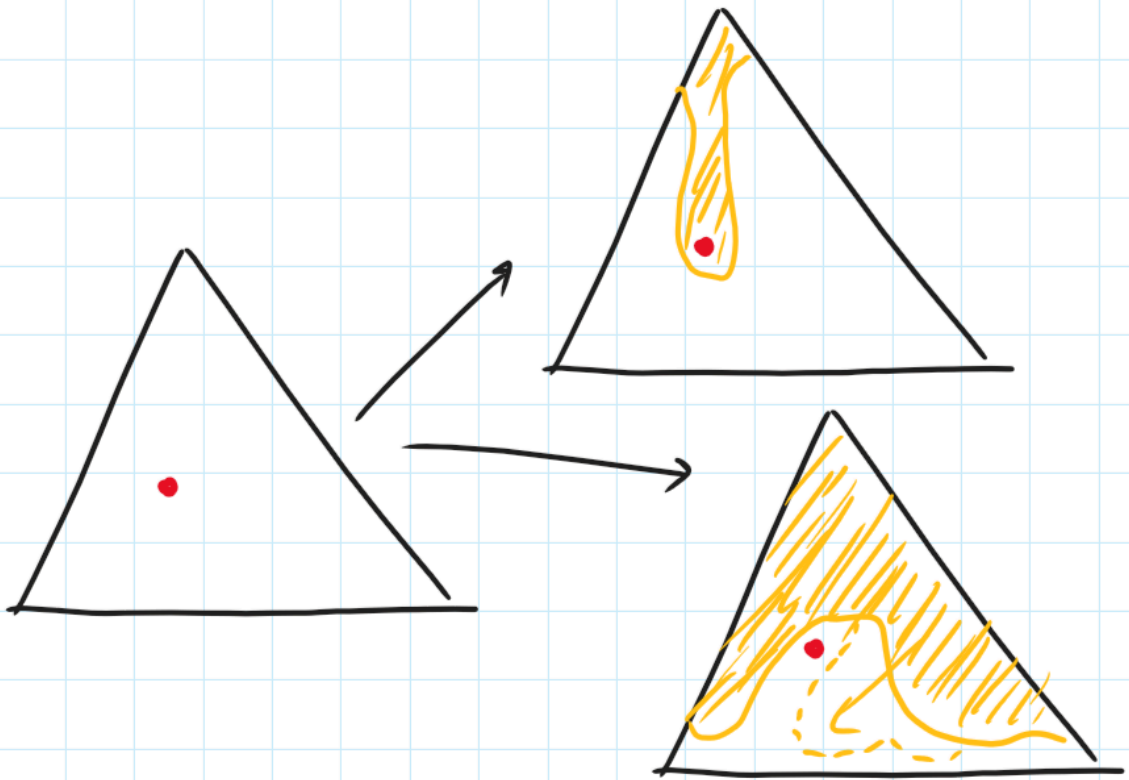
X

2. Greedy Search and A* Search

ALGORITHM. (*Greedy Search*)

Greedy search ignores the arc cost and chooses the next action in such a way the next state minimizes the heuristic cost h . In a sense, it can be thought as a "*guided DFS*".

However, this works only if the heuristic is really good; in some pathological cases, it can even worsen the performance and search time (although it is usually fast).



So we want to combine the "*speed*" of Greedy Search and the *optimality* of UCS: to do it, we simply define the cost function

$$f(n) := h(n) + g(n)$$

Where $h(n)$ is the *forward cost* and $g(n)$ is the *backward cost*.

ALGORITHM. (*A* Search*)

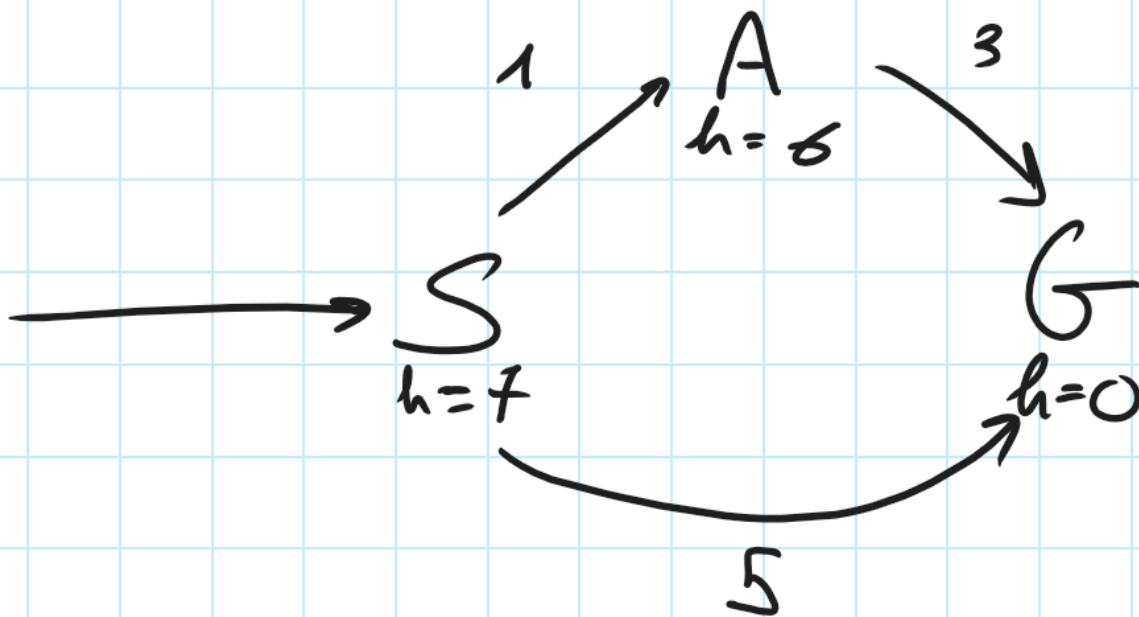
The A* search algorithm is the same as greedy search, except that it maximizes f defined above (^2860af).

X

3. Admissible Heuristics

Q. Can *A* search* ensure optimality?

In some cases, A* is not optimal. Take for instance the following graph search problem:



With A*, the path taken would be (S,G) which has real cost 5, which is not optimal. This is due to the fact that the heuristic has overestimated some costs (namely from S to A and A to G).

More intuitively, we have another example of a heuristic that's not guaranteed to lead optimality; let $h(n) := 1 - g(n)$, then $f(n) = 1$ for every node and so it behaves like a BFS or DFS algorithm.

To avoid this, we want *admissible heuristics*. Let us define an admissible heuristic as follows:

#Definizione

Definizione 11 (admissible heuristic).

A heuristic $h(\bullet)$ is said to be *admissible* if, given the "true cost" h^* , the following inequality is satisfied:

$$0 \leq h(n) \leq h^*(n)$$

To come up with admissible heuristics, we can have a standard procedure.

IDEA. Given a certain problem P_1 modelled through the PEAS framework, let us define its *relaxed problem* P_2 as the problem with the same states but more actions; that is

$$\forall s, \mathcal{A}_2(s) \supseteq \mathcal{A}_1(s)$$

The heuristic given by calculating the *true costs* for the relaxed problem is surely to be admissible. In fact, we can prove that

$$\forall s, h_2^*(s) \leq h_1^*(s)$$

Intuitively, this is due to the fact that if the new actions are "*worse*" then the algorithm will simply take the actions of the old problem.

In a sense, we are trying to "add" some shortcuts in our search graph, which might or might not help our algorithm to find a faster solution.

That said, we can conclude this section by stating the main result on admissible heuristics:

#Teorema

Teorema 12 (admissible heuristics are optimal).

Given an admissible heuristic h w.r.t to a search problem P , the A* tree search algorithm is optimal with the heuristic h .

#Dimostrazione

PROOF of

In general, let A, B be the goals of a search problem and A the solution with optimal path. Suppose that a certain node, ancestor of A , may *prefer* to expand to a path leading to B . Then proving optimality is equivalent to proving

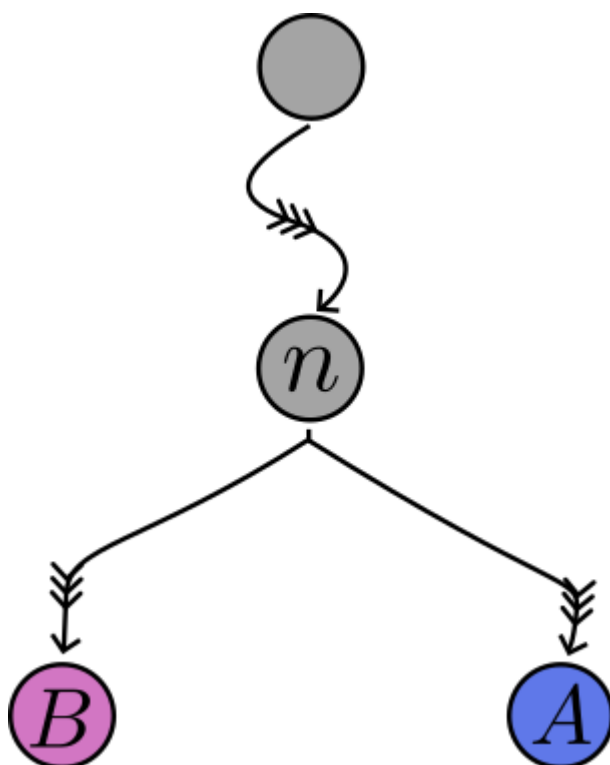
$$f(n) \leq f(A) \leq f(B)$$

In the sense that the algorithm will prefer to expand the nodes leading to A in any case.

Proving $f(A) \leq f(B)$ is trivial, as $g(A) \leq g(B)$ by definition. Let us prove the crucial part of the theorem, that is $f(n) \leq f(A)$. By definition we have that

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n)$$

Noting however that $g(n) + h^*(n) = f(A)$, we can conclude the proof. ■



4. Comparing Heuristics

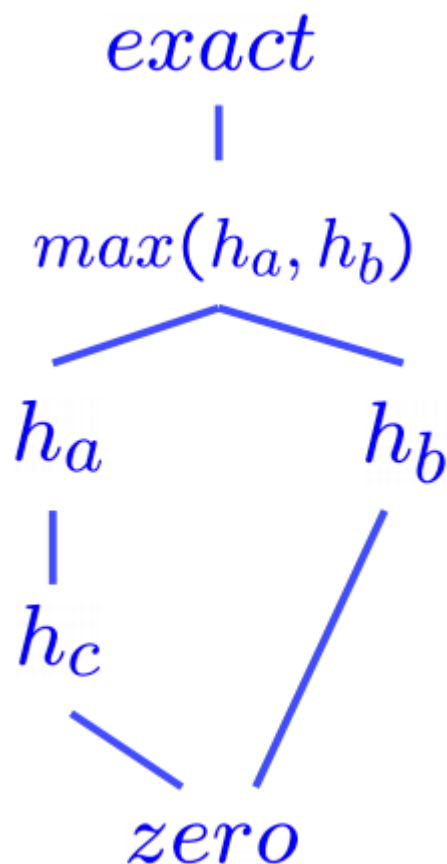
Q. How can we tell if one heuristic is "*better*" than the other one? Can we combine heuristics? What is the trade off in choosing heuristics?

The standard metric for comparing heuristics is *dominance*. Given two heuristics h_a, h_c we can say that $h_a \geq h_c$ if and only if $\forall n, h_a(n) \geq h_b(n)$.

Let us define the trivial heuristic $h_0(n) := 0$; we can see that A* with this heuristic leads to the UCS algorithm. All admissible heuristics dominate the trivial heuristic.

Moreover, we have the following rule: the maximum of two admissible heuristics is admissible.

With these rules, we can build a "*hierarchy*" of heuristics and draw the following semi-lattice:



5. Consistent Heuristics

Suppose we want to use *graph-based* search algorithms instead. In this case, admissibility won't be enough to ensure optimality of the A* graph search algorithm.

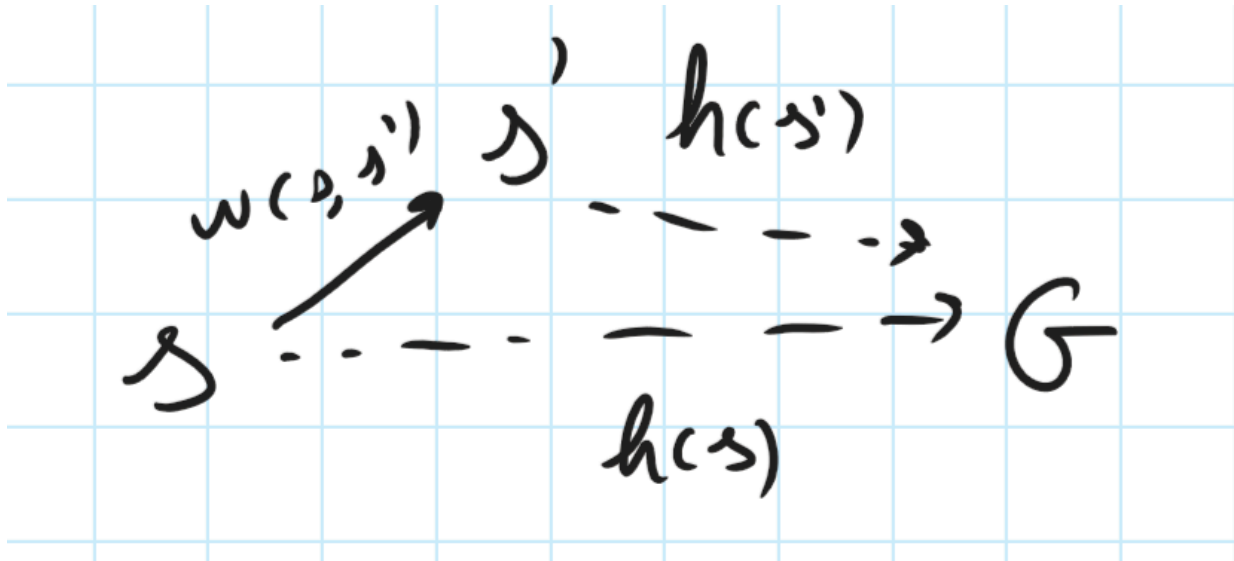
Let us define the consistency of a heuristic as follows:

Definizione 13 (consistent heuristic).

Given a weighted graph $G = (V, E, \omega)$ and a heuristic h , h is said to be *consistent* if the following inequality is satisfied for every legal state pair s, s' :

$$h(s) - h(s') \leq \omega(s, s')$$

In a sense, it satisfies a form of the triangle inequality:



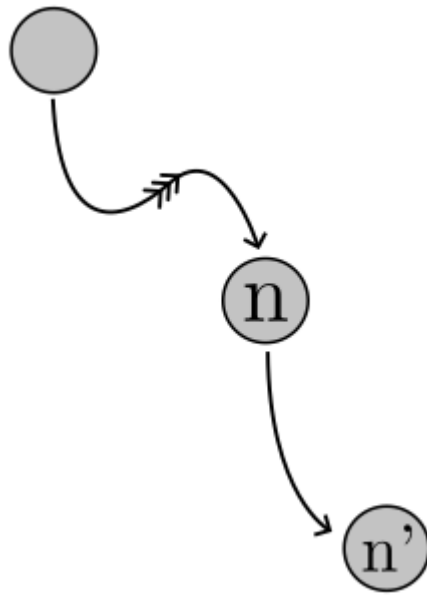
Teorema 14 (consistency implies optimality).

If h is consistent with respect to the weights $\omega(\bullet, \bullet)$ then the A^* graph search algorithm returns an optimal solution.

PROOF of ^55b4ee (*Idea*)

First of all we notice that for a consistent heuristic, the cost is *non-decreasing* for "*deeper*" solutions. Let n' be a node and n a direct ancestor of n' : then $f(n') \geq f(n)$. We can prove this by making a few calculations:

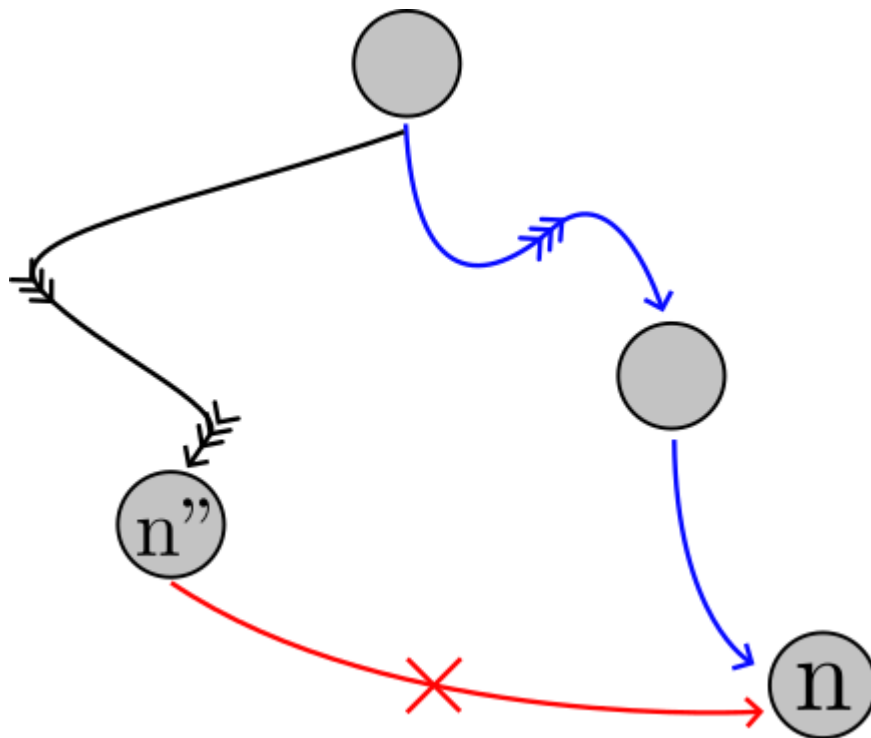
$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= \omega(n, n') + g(n) + h(n') \\ &\geq g(n) + h(n) - h(n') + h(n') \\ &= f(n) \end{aligned}$$



Now we can start the actual sketch the proof.

From the previous fact we can prove that whenever a node n is removed for expansion, then an optimal path leading towards n has been found. If we assume, by contradiction, that is false then we would have some ancestor of n , called n'' , on the frontier that was *optimal* but never expanded ("stuck" on the frontier).

However, this is a contradiction as the values of f along a path are *non decreasing* and so n'' would have already been expanded before, concluding.



Applying this principle to the goal node, we conclude. ■

Relation between Admissibility and Consistency

X

Useful remarks on the relationship between admissibility and consistency of a heuristic. Lemma 1: consistency implies admissibility. Counterexample: admissibility does not imply consistency.

X

0. Voci correlate

- Informed Search Algorithms

1. Relationship between Admissibility and Consistency

Q. What can we say about the consistency of an admissible heuristic and viceversa?

A full answer to this question is given as follows.

#Lemma

Lemma 15 (consistency implies admissibility).

All consistent heuristics are admissible, that is

$$\forall (s, a, s'), h(s) - h(s') \leq \omega(s, a, s') \implies \forall s, h(s) \leq h^*(s)$$

#Dimostrazione

PROOF of

Proof by induction. The main idea is to prove that for the "closest node" to the goal the heuristic is admissible, and so by induction the other nodes must be "admissible" as well by consistency.

Base case: Consider s the node before the goal node, that is a graph of type $S \rightarrow G$. By the definition of consistency

$$h(s) \leq \omega(s, \bullet, G) + h(G)$$

However by definition the heuristic of a goal node is zero, thus

$$h(s) \leq \omega(s, G)$$

And since that $\omega(s, G)$ represents the "true cost" for reaching the goal state, it matches the optimal heuristic:

$$h(s) \leq \omega(s, G) = h^*(s)$$

Inductive case: Consider two neighbouring nodes s, s' such that $h(s') \leq h^*(s)$ (in other words, it's "admissible" there); so we are considering something like this: $s \rightarrow s' \rightsquigarrow G$.

By consistency we have again

$$h(s) \leq \omega(s, s') + h(s')$$

However $h(s') \leq h^*(s')$, so

$$h(s) \leq \omega(s, s') + h^*(s)$$

However $\omega(s, s') + h^*(s)$ is the "*true cost*" of reaching G from s , meaning that it's exactly equal to $h^*(s)$, yielding

$$h(s) \leq h^*(s)$$

Concluding.

#Esempio

COUNTEREXAMPLE. The same cannot be said for the other direction, as admissibility does not imply consistency. Consider the following graph:

$$s \xrightarrow{1} s' \xrightarrow{3} G$$

We can assign the following admissible heuristic: $h(s) = 4, h(s') = 0$ (since that it matches the "*true*" cost for s and it's underestimated for s'), however it's not consistent as

$$h(s) - h(s') = 4$$

Which is greater than the arc cost $\omega(s, s') = 1$.

Heuristics Examples

X

Examples of problems and heuristics. Pancake problem and 8-block puzzle.

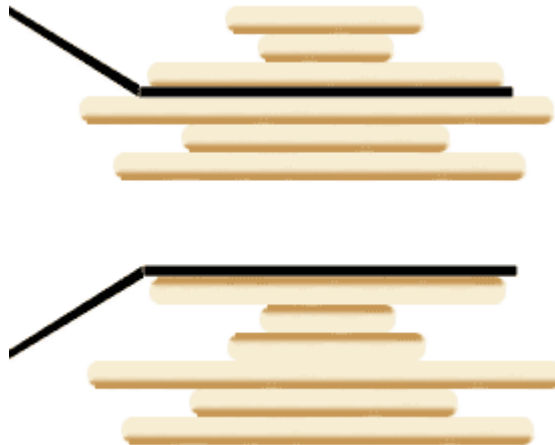
X

0. Voci correlate

- Informed Search Algorithms

1. Pancake Sorting

Pancake sorting is the problem of sorting a "disordered stack of pancakes", and one can "insert" a spatula in a certain layer of the stack and flip all the pancakes above it.



The *string* equivalent of the problem is the *prefix reversal problem*.

It can be proven that the problem is *NP-hard*, therefore it will be "hard" to find an algorithm which solves this in polynomial time.

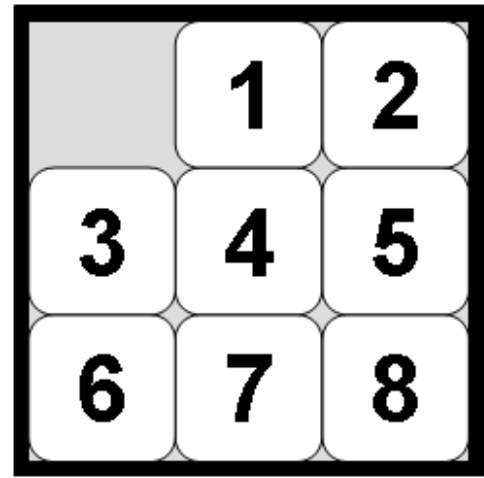
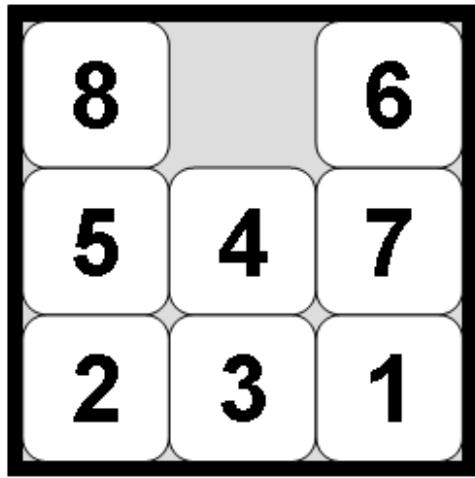
An approach is to use an *informed search algorithm* with an admissible heuristic.

In this case, an *admissible heuristic* consists of counting the number of largest misplaced layers, starting from the bottom.

X

2. 8-block Puzzle

The 8-puzzle consists of an earea divided into a 3x3 grid, where eight tiles are numbered from 1 to 8 and the remaining tile is empty. The objective is to reach a specific configuration of the tiles, which is usually the sorted grid with the empty tile in the top-left corner.



Let us do a preliminary analysis of the problem.

State Space: The whole problem can be encoded as a vector, so the size is 9!

Actions: We can swap the empty tile with one of its neighbouring squares

Costs: Uniform

Now, if we want to use an informed search algorithm, we can use the *heuristic* which consists of *counting the amount of misplaced tiles*.

It is *admissible* as the heuristic represents the costs of a *relaxed problem*, that is blocks can be swapped freely with each other.

Local Search Algorithms

X

Introduction to local search algorithms and optimization. Hill Climbing, Simulated annealing (properties), Beam Search and Genetic Algorithms.

X

0. Voci correlate

- Definition of Search Problems
- Cenni alla Ricerca Locale e Metaeuristiche

1. Hill Climbing Algorithm

Local search algorithms begin from a start state and move into its neighbours without keeping track of the path.

They work by attempting to find solutions for *optimization problems* where the *best state* is determined according to a certain *objective function*. In a certain sense, we have an "*initial guess*" and we want to improve it, eventually reaching a "*good solution*".

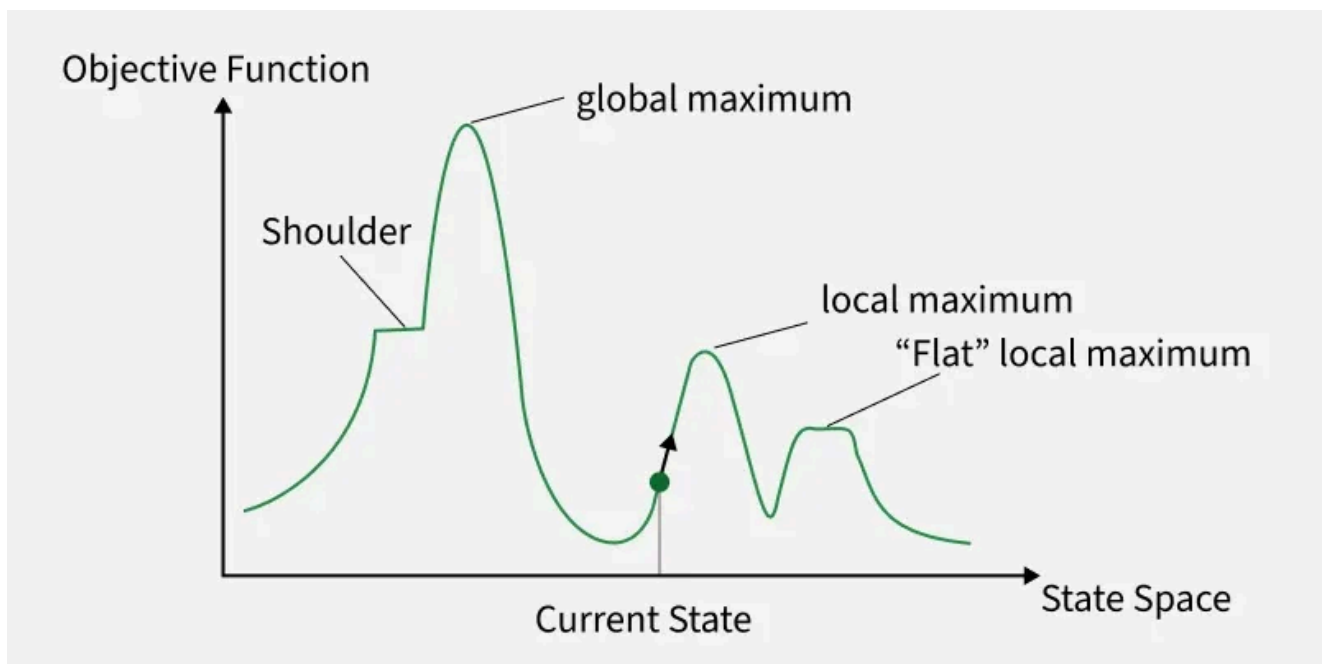
The best scenario to use local search is one where we can have a good "*initial guess*" to begin with!

The first and most intuitive algorithm is the *hill climbing algorithm*, which works by exploring a state's neighbour and greedily considering its best neighbour. So the pseudocode is simply as follows:

1. Let S_0 be the initial state, $\text{neigh}(s)$ the function which returns the neighbours of a certain state s and ν the objective functions (wlog we assume that it's to be maximized).
2. Let *cur* be the *initial state* S_0
3. Loop:
 1. Let *neigh* be $\arg \max_{s' \in \text{neigh}(\text{cur})} \nu(s')$
 2. If $\nu(\text{neigh}) \leq \nu(\text{cur})$:
 1. Return *cur*, as we do not have "*better solutions*" anymore
 3. Else:
 1. Let *cur* be *neigh*

The algorithm is very simple to implement, however with a caveat; the algorithm does not insure completeness or optimality at all, since that it can get easily stuck in local or "*flat*" minimas.

Some variants attempt to fix the problem, such as the "*random restarts*" variant of hill climbing; in this case it picks s_0 randomly.



Another popular variant of this is the *gradient descent*, where we minimize a function instead of maximizing it.

X

2. Simulated Annealing

The *simulated annealing* algorithm takes inspiration from *statistical physics*, in particular with respect to the *annealing process used to cool metals slowly to reach an ordered (low-energy) state*. Here, other than defining an initial state we also define a *schedule*, which is a function valued on time which is initially "*tolerant*" towards taking worse solutions, and then becomes more "*rigid*" as time passes. In a way, the schedule defines the "*temperature*" of the algorithm in a certain time step.

Here is the pseudocode as follows:

1. Let s_0 be the initial state, σ the schedule
2. Let *cur* be s_0
3. For t looping from 1 to $+\infty$ (or a manually set threshold $\hat{T}$):
 1. Let T be $\sigma(t)$
 2. If $T = 0$: return *cur* as the solution.
 3. Else:
 1. Let *next* be a randomly picked neighbour of *cur*
 2. Evaluate the difference in energy ΔE as $\nu(\text{next}) - \nu(\text{cur})$
 3. If $\Delta E > 0$: let *cur* be *next* and continue
 4. Else: let *cur* be *next* with probability $\exp(\Delta E/T)$

Let us make some remarks on the solution:

#Osservazione

REMARK 1. (*Probability Properties*)

First of all the probability $\exp(\Delta E/T)$ is well defined, since that $\Delta E < 0$ in the cases where it's calculated so surely it's in the range $(0, 1)$. Moreover, the amount is *decreasing* for a tigger t : intuitively, the temperature "*cools down*" with increasing time so the quantity vanishes to 0.

Moreover, using Markov Chain Analysis we can also prove that the *probability* of the algorithm visiting a certain node x is proportional to it's "*energy*":

$$p\{s = x\} \propto E(x)$$

Meaning on the *long run* the algorithm will prefer to visit the state with the largest energy. This leads us to the final result:

#Teorema

Teorema 16 (optimality of Simulated Annealing).

Given the *Boltzmann distribution*, we have that the Simulated Annealing algorithm converges to the *optimal state* if T converges towards 0 "*slowly enough*".

#Dimostrazione

PROOF of $\wedge$ 9fc71d (*Idea, optional*)

Consider two adjacent states x, y with $E(y) > E(x)$. Assume that the transitions $x \rightarrow y$ and $y \rightarrow x$ are legal, with the same outdegrees d . Given the balance condition of the state

$$p\{s = x\}p\{s' = y \mid s = x\} = p\{s = y\}p\{s' = x \mid s = y\}$$

We have that

$$p\{s = x\} \frac{\exp(\Delta E/T)}{d} = p\{s = y\} \frac{1}{d}$$

So we have the proportion

$$\frac{p\{s = x\}}{p\{s = y\}} = \frac{\exp(E(x)/T)}{\exp(E(y)/T)}$$

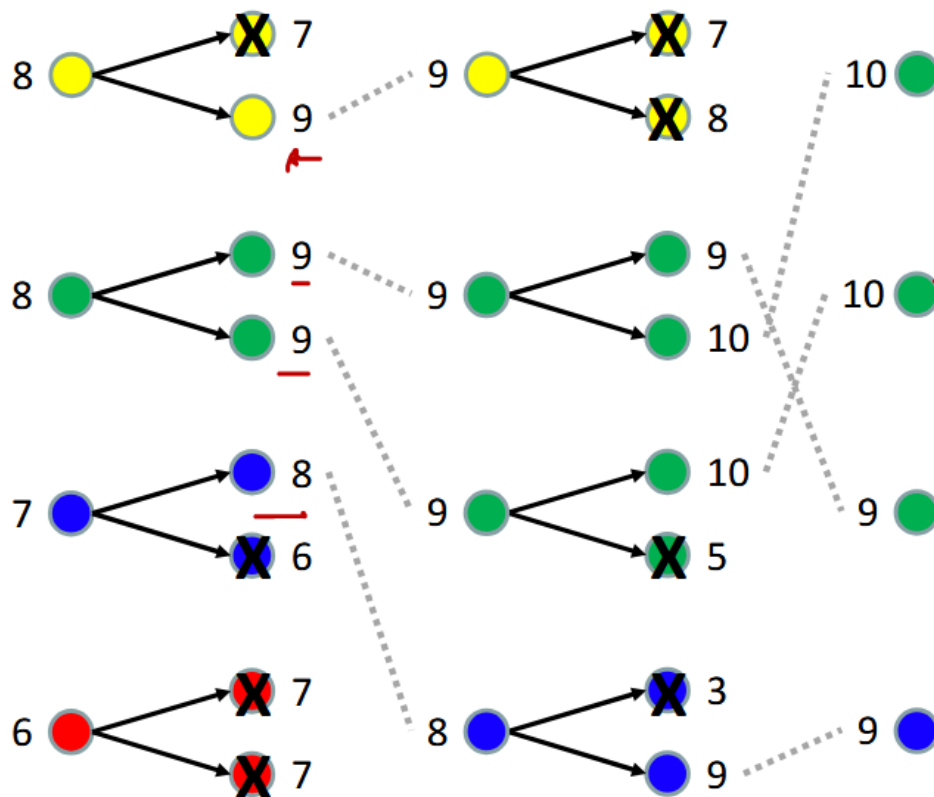
So for $T = 0$ we only take the local best, when $T \rightarrow +\infty$ we take with equal probability any possible step. ■

So we conclude by stating that *simulated annealing* is a convergent algorithm, albeit "*slowly*" convergent.

X

3. Local Beam

Idea. Make k copies of a Local Search algorithm, and let them "*reproduce*" by exploring the current state. Then pick the k -best "*solution proposals*".



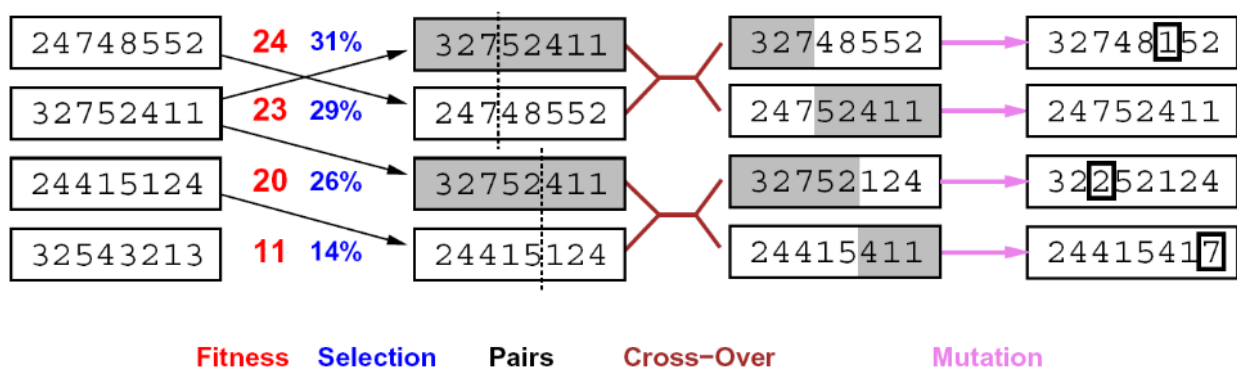
The only problem is a lack of diversity in the proposed solutions. To avoid this, we have the variant *stochastic beam search*, which randomly selects successors proportional to their value.

X

4. Genetic Algorithm

IDEA. Like before, we want to consider "*multiple*" instances of local search algorithms. However, here not only the instances are *parallel* but they also "*communicate*" with each other. In addition, we consider two more factors:

- "*Cross Over*": The proposed solutions will "*combine*" with each other with pairwise crossover operators
- "*Mutation*": Apply single-point mutations to the solutions, to add a stochastic factor



Logics and Constraint Satisfaction Problems

Logics for AI

X

Recap of logics for AI.

X

1. Entailment

#Definizione

Definizione 17 (entailment).

We say that a formula φ *entails* ψ if and only if in *every model* where φ is true, ψ is also true. In the case of an entailment we denote it with $\varphi \models \psi$.

More generally, $\varphi \models \psi$ iff $\varphi \implies \psi$ is *valid*, that is $\neg\varphi \vee \psi$ is *valid*.

X

2. Proofs

#Definizione

Definizione 18 (proof, soundness and completeness).

A *proof* is a demonstration of entailment between formulas α, β .

An *algorithm* is sound if everything it claims is entailed. It is *complete* if everything that can be entailed is proven.

Two ways to provide *proofs*:

- *Model Checking*: Check for *every possible world* that if α is true then β is true too (propositional logic).
 - For example, we can use the *Tableau* (or *DPLL*) method on the negation and see whether it is unsatisfiable or not (if it is, then it means the entailment is true as the implication would be valid)
- *Theorem Proving*: Search for a sequence of proof steps (inference rules) leading from α to β .
 - For example, the *resolution algorithm* applies iteratively the following three rules:
 - From $A, A \implies B$ we can infer B (*modus ponens*)

- From $A \wedge B$ and the fact we can infer A , we can also infer B . (*and-elimination*)
- From A, B we can infer $A \wedge B$ (*resolution*)
- Or the *forward chaining* algorithm can be applied in the special case we *only* have literals and implications.

3. Knowledge-Based Agents (an idea)

An agent can be designed through *logics*, in particular it is made out of two components:

- *Knowledge base*: the "axioms"
- *Inference engine*

Example. Pacman can be coded as a KB-agent, where:

- The *Knowledge Base* defines the map, the physics of the game, et cetera...
- The *Inference Engine* encodes the history of the agent, in particular the successor-state action

The most important part of the inference engine is the *state-estimation* ability, that is to "figure out what state the agent is in".

Generalmente, the *successor-state* action is coded as follows:

$$x_t \iff ((x_{t-1} \wedge \varphi) \vee (\neg x_{t-1} \wedge \psi))$$

Where φ is an "*action which did not cause x_t* " and ψ the "*action caused x_t* ". In a certain sense, the first side of the OR operator represents that the state remained the same, and the other side represents that we are in a different state.

Esempio 19 (example of a successor-state axiom).

An example can be as follows:

$$\text{Hot}_t \iff (\neg \text{Hot}_{t-1} \wedge \text{StepCloseToLava}_{t-1}) \vee (\text{Hot}_{t-1} \wedge \neg \text{StepAwayFromLava}_{t-1})$$

A knowledge-based agent will also *perceive* the environment.

The *reasoning tasks* are what KB-based agents can do, and they are as follows:

- *Localization*: Infer the position of the agent
- *Mapping*: Build a "map" of the environment
- *SLAM*: Localization and Mapping, at the same time
- *Planning*: Find a sequence of actions towards the goal

All of these tasks can use the *same algorithms* and *knowledge base*, in fact it's sufficient to encode the knowledge base in order to "ask" the agent to do all of the tasks above mentioned.

Introduction to Constraint Satisfaction Problems

X

Introduction to constraint satisfaction problems (CSP). Framework for a CSP, concept of identification. Examples: map coding, N-queens and Cryptarithmic. Varieties of variables, constraints. Constraint graphs, examples.

X

0. Voci correlate

- Logics for AI

1. Definition of Constraint Satisfaction Problem

With the *formulation of search problems*, we were looking for a *sequence of actions* which lead us to the state goal. In this case, we are *planning* the path to the goal.

Now we will learn about solving another subset of problems, known as *identification problems*, where instead of focusing on the path, we will focus on the *goal* itself. Moreover, all paths are at the same depth (for some formulations).

A particular class of *identification problems* are the so-called *constraint satisfaction problems*, defined as follows:

#Definizione

Definizione 20 (Constraint Satisfaction Problem).

A *constraint satisfaction problem* is a triple $(\mathcal{X}, \mathcal{D}, \mathcal{C})$ where:

$\mathcal{X} = \{X_1, \dots, X_n\}$ is the *set of variables* and is used to define the *state* of the problem

$\mathcal{D} = \{D_1, \dots, D_n\}$ is a set of *domains*, that is the possible values that can be assigned to each variable X_i .

$\mathcal{C}$ is a *set of constraints* that specify the feasible combinations of assignments to variables. In particular, each constraint is a pair made out of *scope* and *relation*, where the scope is a tuple of variables and relation is a relation on the scope. $\mathcal{C}$ will form the *goal state*

If every constraint in $\mathcal{C}$ relates at most two variables, we can say the CSP is *binary*.

Given a CSP $(\mathcal{X}, \mathcal{D}, \mathcal{C})$ with n variables, we say that an *assignment* $\varphi : V \subseteq \mathcal{X} \rightarrow \mathcal{D}$ is:

- *Consistent* if it does not violate any constraint in $\mathcal{C}$
- *Complete* if for every variable there is an assigned value (i.e. $V = \mathcal{X}$)

An assignment is a *solution* if it's *consistent* and *complete*. Generally, finding a solution to a CSP is a NP-complete task, meaning that it most likely takes an exponential amount of time (see: NP vs P problem).

X

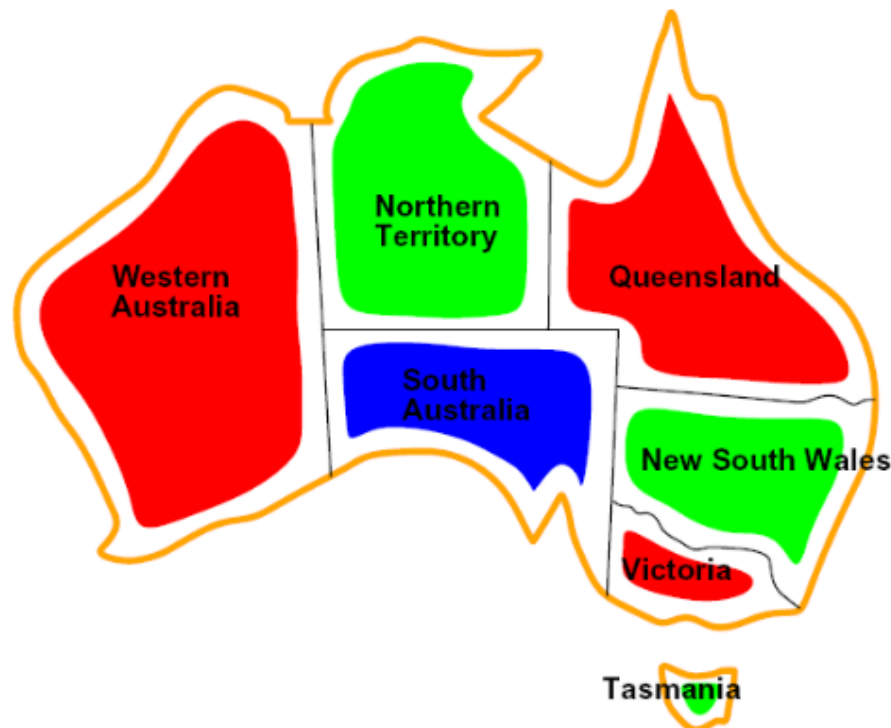
2. Constraint Graphs

CSPs are often represented as *constraint graphs*, where nodes represent variables and edges represent constraints between them. For each "type" of constraint, we draw the edge in a different manner:

- *Unary*: Not represented, they can be used to prune the domain of the variable they constrain when we will solve the CSP
- *Binary*: As a traditional graph edge
- *n-ary*: They can also be represented as *edges*, they just look unconventional. However, it's better to figure a work-around to transform the *n*-ary constraint into multiple binary ones.

2.1. Map Coding

Given a *map* of regions and *three colours*, we want to assign a colour to each region in a way that no region touches a region with the same colour (*graph colouring problem*).



We can code this as a CSP in the following way:

- $\mathcal{X} = \{\text{WA}, \dots, \text{T}\}$ (seven regions)
- $\mathcal{D} = \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}$ (colours coded in RGB triples)
- $\mathcal{C}$: We can either define constraints *implicitly* or *explicitly*.

- Implicitly: $WA \neq NT \neq SA$, et cetera....
- Explicitly: $(WA, NT) \in \{[(1, 0, 0), (0, 1, 0)], \dots\}$

One example of a solution is to assign $WA=red$, $NT=green$, $Q=red$, $NSW=green$, $V=red$, $SA=blue$ and $T=green$.

#Osservazione

Remark. Note how there can be *multiple solutions* to a CSP. In fact, by drawing the CSP graph we notice that there is a node that has no constraints.

EXERCISE. Code the n -queens problem into a CSP

EXERCISE. Code the *cryptarithmic problem* into a CSP, being to figure out what numbers we need to assign to $O + O$, $W + W$, $T + T$ such that $TWO + TWO = FOUR$.

X

3. Varieties of CSPs

For any CSP problem:

- The variables can be either *discrete or continuous*.
 - If discrete, then the domains can be either *finite or infinite*. Note how that if we work with infinite domains, it will be necessary to work with *implicit constraints*.
 - If continuous, then we have *Linear Programming* problems
- The constraints can be either *unary*, *binary* or of *higher order*.
- A constraint can be *soft* if they represent a *cost* for a certain variable assignment, giving us a *constrained optimization problem*.

Solving Constraint Satisfaction Problems

X

Solving CSPs. Standard CSP formulation, idea of the general method. Backtracking search: definition, ideas of improvement.

X

0. Voci correlate

- Introduction to Constraint Satisfaction Problems

1. Backtracking Search

Given a *standard formulation of CSPs*, we can formulate the *problem of finding a solution* φ as the following search problem:

- *Initial state*: empty assignment, $\varphi = \{\}$
- *Successor function*: For each state, its successor is an assignment of a value in $\mathcal{D}$ to any variable X_i that has not been assigned yet
- *Goal test*: Check if the assignment is *consistent*.

However, applying *classic search algorithms* to this particular search problem is a horrible idea, for a lot of reasons.

The *main algorithm* to solve this search problem is as follows:

#Definizione

Definizione 21 (backtracking search).

The *backtracking search algorithm* consists in solving a CSP by considering *single variable assignments* for each step and the values considered are those who *do not conflict with the previous assignments*.

In other words, it's just DFS with two improvements:

- One variable at a time, fix the ordering of variables
- Check constraints *"as you go"*, incremental goal test

PSEUDOCODE. (*Backtracking search*)

- Let $\varphi = \{\}$ be the empty assignment, and $\text{CSP} = (\mathcal{X}, \mathcal{D}, \mathcal{C})$.
- Work recursively, with input φ and CSP , will return either *failure* or *a complete assignment*:
 - If the assignment φ is *complete*, terminate

- Select a *variable* X_i from the *unassigned variables* of the CSP
- For each *value* D_i in the domain values (ordered),
 - If *value* is consistent with the *assignment* given $\mathcal{C}$, then:
 - Extend φ with $\{X_i = D_i\}$
 - Go back to the second point ("*work recursively*") and save its *result*
 - If the result is *not* a failure, then return φ (propagate)
 - Else:
 - Remove the extension $\{X_i = D_i\}$ from φ
- Return *failure*

However, the backtracking algorithm is still not perfect and can be improved. The grounds for improvement are as follows:

- *Ordering*: How should we order the variables? How can it affect the performance?
- *Filtering*: How can we detect failures earlier, rather than waiting for a dead end?
- *Structure*: By analyzing the structure, can we find some exploitable features?

Filtering

X

First way to improve the backtracking search. Filtering: forward checking, and constraint propagation, definition of consistency on nodes, archs and on k variables. Examples.

X

0. Voci correlate

- Solving Constraint Satisfaction Problems

1. Filtering

The first improvement to CSP performance we'll consider is *filtering*, which checks if we can prune the domains of unassigned variables *ahead of time*.

FORWARD CHECKING. A naive method is *forward checking*, which works by assigning $X_i = D_i$ and then pruning the domains of unassigned variables that share a constraint with X_i .

Another approach is *constraint propagation*:

CONSTRAINT PROPAGATION. Forward checking propagates information from assigned to unassigned variables, but still does not provide early detection for all failures. Constraint propagation will *iteratively* enforce the constraints locally.

X

2. Consistency

Let's see how we can realize *constraint propagation*. First let us define the notion of *consistency*.

#Definizione

Definizione 22 (consistency).

A *single variable* is *node-consistent* iff all the values in the variable's domain satisfy the variable's unary domain. The *CSP-graph* is *node-consistent* iff every variable in the graph is node-consistent.

An *arc* (x, y) is *consistent* iff for every value of $x \in X$ there is some allowed $y \in Y$.

A single variable X_i is *arc-consistent* iff every value in its domain satisfies the variable's binary constraints, that is every arch of type $(X_i, \bullet)$ is consistent.

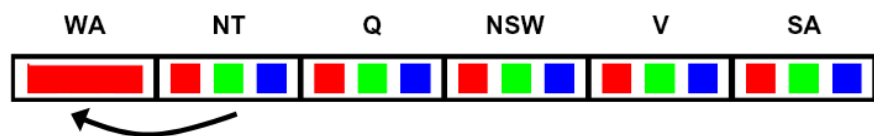
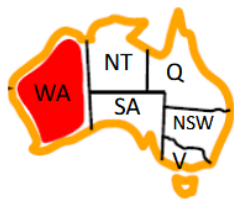
Two variables X_i, X_j is *path-consistent* w.r.t. a third variable X_k iff every consistent assignment $\{X_i = a, X_j = b\}$ with the constraints on X_i, X_j there is an assignment to X_k that satisfies every constraints between X_i, X_k and X_j, X_k .

More generally, a CSP is *k-consistent* iff for any set of $k - 1$ variables and for any consistent assignment to them there can be a consistent value assigned to any k -th variable.

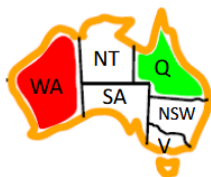
Note that *node, arc, path* consistency is just *k-consistency* for $k = 1, 2, 3$.

Example. Let us see an example with the *map colouring problem*. Suppose we want to enforce the arc consistency of the CSP, then:

- *Step 1.* Suppose we assign WA=red. Then, we want to check that (NT, WA) is consistent:
 - If NT=blue: OK
 - If NT=green: OK
 - If NT=red: Not consistent
- Then if we remove the assignment NT=red, we have the arc-consistency (NT, WA). Then repeat the same procedure for all other couples. In the end, if we do not have problems (empty domains), we can proceed with *forward checking* and prune domains.



- *Step 2.* Suppose now we assign Q=green. Can we enforce arc consistency? Let us prune the domains first, note however that (SA, NT) to be consistent we must prune SA's domain to an empty set. This makes us detect a *failure*, making us exclude the assignment just made,.



So *constraint propagation* is a procedure that can be either run as a *preprocessor* or as a *"soundness check"* for each assignment made.

The procedure described with the example can be generalized with the *AC3 algorithm*, with time complexity $O(|E|d^3)$ where d is the size of the largest domain.

```

function AC-3(csp) returns the CSP, possibly with reduced domains
inputs: csp, a binary CSP with variables  $\{X_1, X_2, \dots, X_n\}$ 
local variables: queue, a queue of arcs, initially all the arcs in csp

while queue is not empty do
     $(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$ 
    if REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) then
        for each  $X_k$  in NEIGHBORS[ $X_i$ ] do
            add  $(X_k, X_i)$  to queue

```

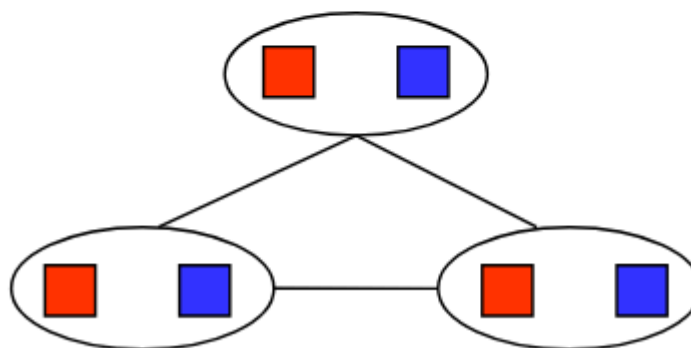
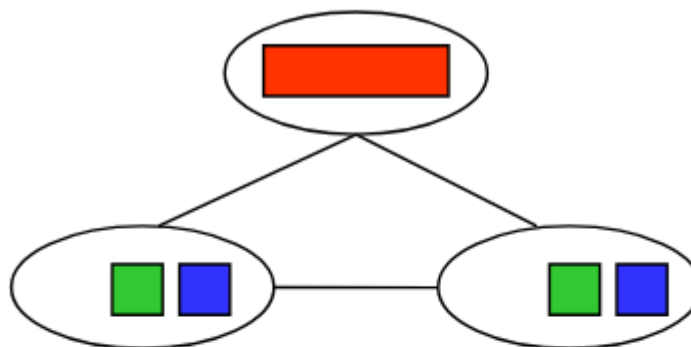
```

function REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) returns true iff succeeds
    removed  $\leftarrow$  false
    for each  $x$  in DOMAIN[ $X_i$ ] do
        if no value  $y$  in DOMAIN[ $X_j$ ] allows  $(x, y)$  to satisfy the constraint  $X_i \leftrightarrow X_j$ 
            then delete  $x$  from DOMAIN[ $X_i$ ]; removed  $\leftarrow$  true
    return removed

```

Arc consistency still has some limitations, in fact sometimes we can:

- Have one solution left
- Have more solutions left
- Have no solutions left (and do not know it)



VALUED

In fact, we still need to run this method in a *backtracking search*.

More generally, we have *strong k -consistency*:

#Definizione

Definizione 23 (strong k -consistency).

A CSP is *strongly k -consistent* iff it's also $k - 1, k - 2, \dots, 1$ -consistent.

Note that if a CSP is strongly n -consistent (where n is the number of variables), then it can be solved without backtracking (directly!). In fact, we can incrementally choose variables with consistent choices with the previously-chosen variables.

However, establishing a strongly n -consistent CSP costs exponential time. A "*good middle ground*" is $k = 3, k = 2$ with the "*weak*" consistency.

Ordering and Structure

X

Other ways of improving backtracking search. Ordering: MRV, LCV. Structure: notion of independent subproblem, tree-structured CSPs, cutset conditioning and tree decomposition.

X

0. Voci correlate

- Solving Constraint Satisfaction Problems
- Filtering

1. Ordering

Let us look at a second approach in *improving* the backtracking algorithm. With *ordering*, we concern ourselves to make a "*sensible*" choice of variables to assign first.

Up until now, we either used a *fixed order* or *random choice*, neither of which are optimal. Intuitively, we want to choose *variables* that has the most "*propagation*" and the *values* that has the least "*constraining force*".

MRV. The heuristic that consists in choosing the variables with the *fewest legal values* is called "*minimum remaining values*" principle, and the idea behind it is to choose the variable that's more likely to fail, pruning the tree earlier.

LCV. Having chosen a variable, we must also choose a value for the variable to assign. We prefer to *rule out the fewest choices* for the neighbouring variables in the constraint graph (i.e. prune the fewest values from the domains of the remaining variables); in this case, we are using the *least-constraining value* principle. Notably this requires additional computation for rerunning arc consistency/forward checking for finding the LCV of the CSP.

X

2. Structure

When approaching a *CSP* in its graph form, it can be useful to look at its *structure* to devise an efficient strategy. In particular, we want to observe:

- *Disconnected parts*: we can treat them as sub-problems
- *Nodes connected to every other node*: we can treat them as "*parent*" nodes, creating some sort of hierarchy

2.1. Independent Subproblems

#Definizione

Definizione 24 (independent sub-problems).

An *independent subproblem* is identified as the connected component of the CSP graph.

The union of the solutions for each independent subproblem will lead to a *solution* for the *complete problem* (we can see intuitively why...). Moreover, the time complexity is also improved, as we have $O((n/c)d^c)$ where c are the variables for the subproblems.

2.2. Tree-Structured Graphs

Unfortunately, CSPs are not structured in independent subproblems (or it's very unlikely to be)

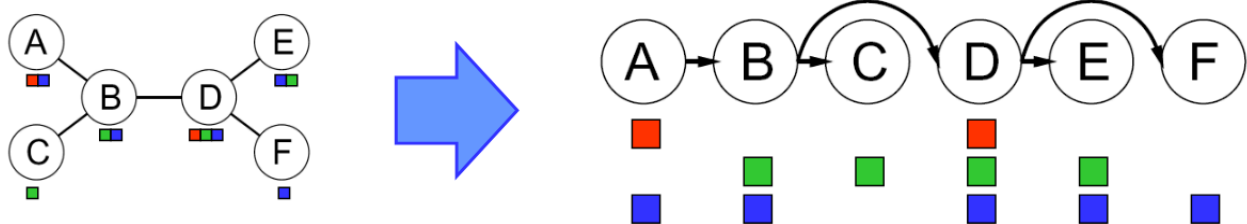
So let us take another approach.

Review. We say that a graph is a tree iff it's acyclic.

CSP graphs which are also *trees* are interesting to us as they can be solved in *linear time* in the number of variables, $O(nd^2)$. In fact, given a CSP tree we can use the following algorithm:

1. *Order*: randomly pick a root variable and order the variables by precedence (parents precede children)
2. *Remove backward*: enforce directed arc consistency starting from the *parents* of the *rightmost children* and going backwards
3. *Assign forward*: starting from the leftmost node assign the values consistently

Note that the step 1. is just the topological sorting of an acyclic graph.



#Teorema

Teorema 25 (soundness and completeness).

The algorithm described above is *sound* and *complete* and runs in time $O(nd^2)$.

#Dimostrazione

PROOF of

Claim 1. After backward pass, all root-to-leaf arcs are consistent

Each arch $X \rightarrow Y$ is consistent as at one point it was made consistent and Y 's domain could not have been reduced thereafter, since its children were processed before.

Claim 2. If root-to-leaf arcs are consistent, then forward assignment will *not* backtrack

We can prove this claim by induction on the position on the nodes.

Claim 3. The algorithm runs on time $O(nd^2)$

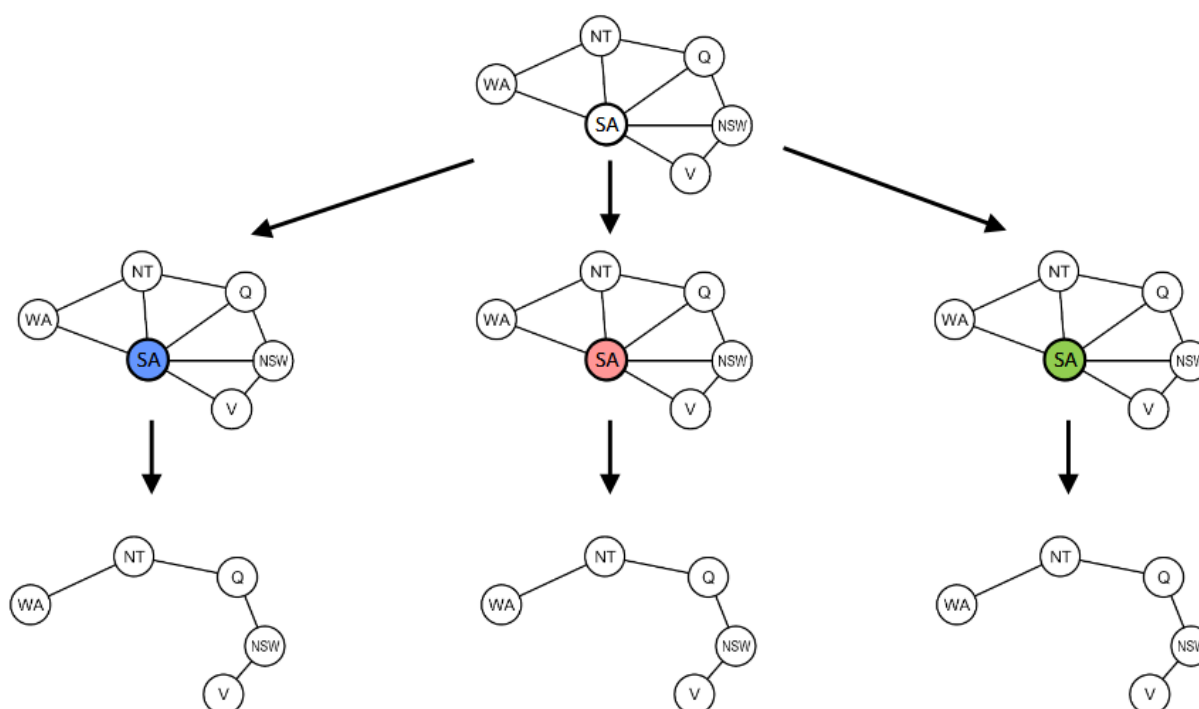
Step 1. has runtime $O(V + E)$, as we will need to use variants of BFS/DFS to convert a DAG to a tree. Then Step 2 runs in time $O(nd^2)$ as we have n nodes multiplied by two possible choices in domains. Step 3. is even faster, as we are doing direct assignments (so $O(nd)$). Overall, $O(nd^2)$ dominates, concluding the proof. ■

2.3. Conditioning

Unfortunately, not all graphs are *DAG*. However, by *modifying* the graph we can make it into a *tree*.

The approach is called *cutset conditioning*, which consists of:

- Choosing a *cutset* (set of nodes)
- Instantiate the *cutset* in all possible ways
- Compute residual CSP for each assignment, so prune the domains
- Solve the residual CSPs



Assuming the *cutset size* c , the algorithm of this is $O((d^c)(n - c)d^2)$. In other words, we should choose the *smallest cutset* possible.

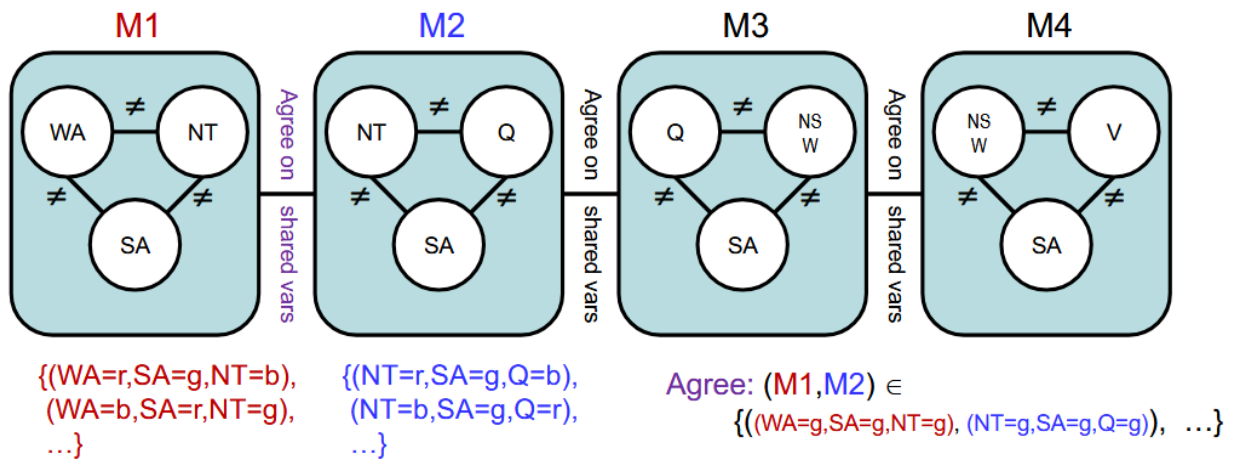
2.4. Tree Decomposition

Another way (and more generally) of transforming a graph into a tree is by encapsulating multiple variables into single nodes. This process is called *tree decomposition*. For it to be applied, the following assumptions must be followed:

- Every variable in the original problem appears in at least one of the tree nodes

- If two variables are connected by a constraint in the original problem, they must appear together in atleast of the tree nodes
- If a variable appears in two nodes of the tree, it must appear in every node along the path connecting those nodes (the problems must *"overlap"*)

Each nodes of the tree is a *"megavariabale"*, where the domain is a *tuple of values*. We need to choose the tuples that preserve arc consistency.



Local Search on Constraint Satisfaction Problems

X

Local search on CSPs problems. Typical scenario, min-conflict heuristic. Critical ratio.

X

0. Voci correlate

- Introduction to Constraint Satisfaction Problems
- Local Search Algorithms

1. Local Search for CSPs

Suppose we somewhat have a complete *assignment* φ on a certain constraint satisfaction problem, assume it is illegal. We can use *local search algorithms* to reassign the values of a randomly chosen variables (which "*breaks*" one of the constraints).

In particular, to choose the *value* to reassign we can use the *min-conflicts* heuristic to choose the value that results in the minimum number of conflicts with other variables.

So our local search algorithm can have the following pseudocode:

- While not solved,
 - Select variable X_i that is conflicting randomly
 - Choose the value D_i to reassign with the *min-conflicts heuristic*

In other words, we're doing hill climb with $h(n)$: total number of violated constraints.

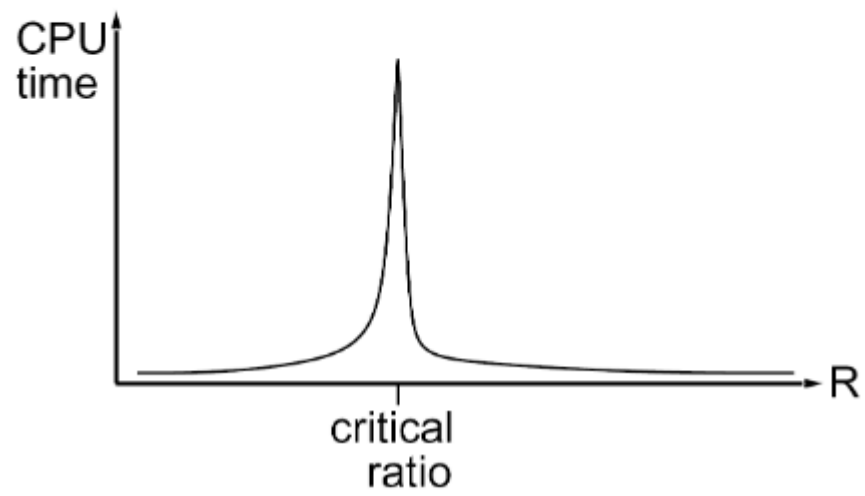
#Osservazione

Remark. Let us observe the performance of this algorithm. Usually, given a random initial state we can solve a problem in *almost constant time* with *high probability*. However, that is not always true, in particular for a certain set of initial states. If we define R as the ratio between then number of *constraints* and the number of *variables*, we have a certain *critical ratio* where using local search with minconflicts becomes extremely expensive.

Moreover, the algorithm used is not guaranteed to be *complete* neither to be *optimal*.

-

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$



Games and Adversarial Search

Definition of Games

X

Introduction to Games and Adversarial Search: definition of a game, intuitive examples. Types of games. Intuition behind the term "zero sum" game. Single-agent games.

X

0. Voci correlate

- Definition of Search Problems

1. Introduction to Games

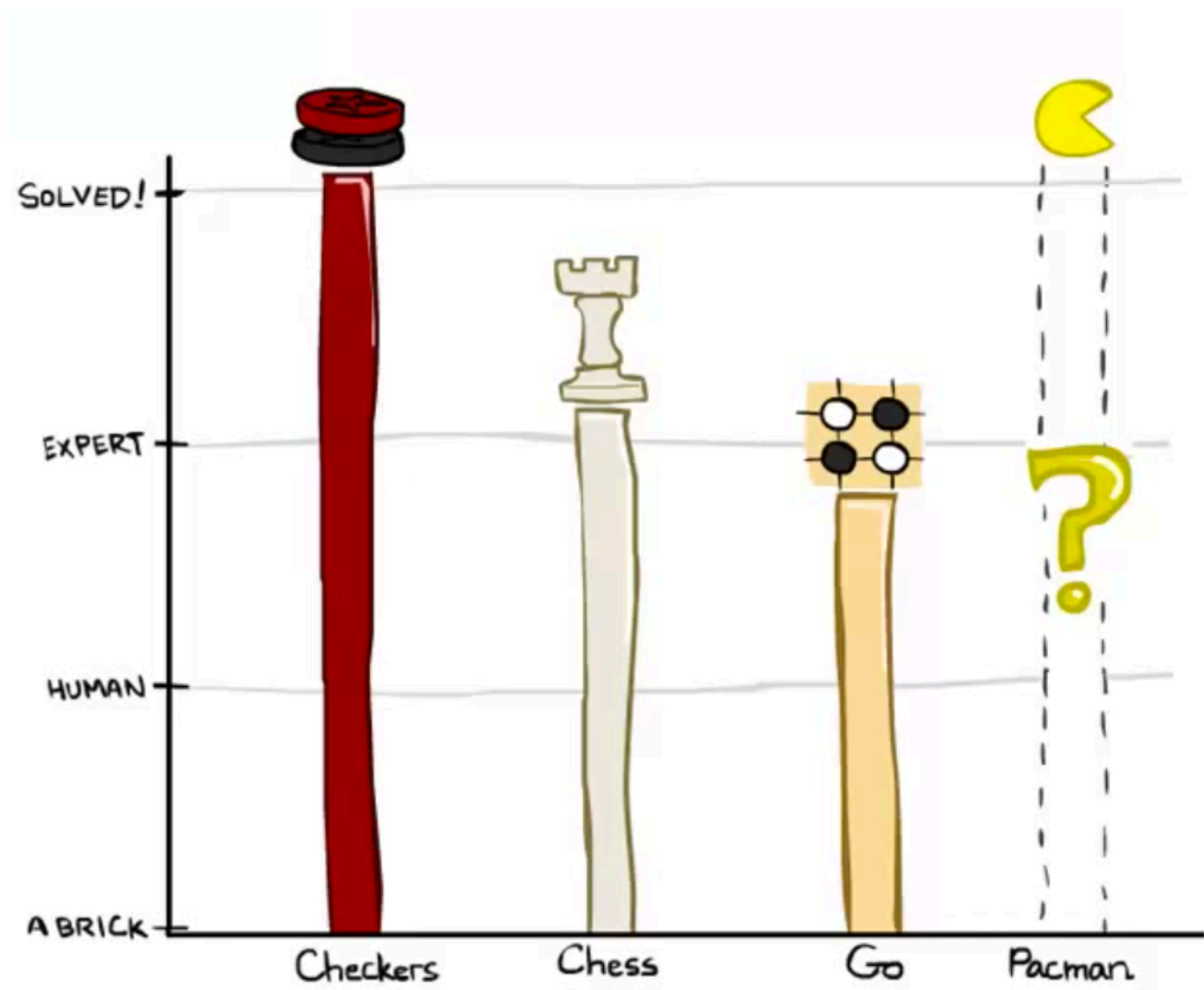
Up until now, we have talked about *search problems* and how to solve them with various search algorithms, spanning from the "classic" techniques such as A*, to more "modern" approaches such as genetic approaches, or with a logic representation.

Now we will explore another class of problems, called *"games"*; they no longer involve in only a *single agent* reaching a certain goal, but rather we will have *more agents* competing each other to achieve their goals.

Examples of such games are *Tic Tac Toe*, *Chess*, *Go* and so on...

The intuition behind a *"game problem"* is that we have an evaluation function of a final state, and the best outcome for each initial state is *"If I play rationally"* from that point on; we will see that we can formalize games through *trees*.

Each game have their own *"level of complexity"* (in poor words, it's the branching factor) and some of them were fully solved, others were not.



So we can classify "*games*" into more categories, such as:

- *Amount of players*: from 1 to n , usually we will talk about *two-player* games ("adversaries")
- *Determinism* or *Stochasticity*: whether actions have deterministic or random outcomes
- *Information Availability*: in a certain sense, the environment can be either fully or partially observable

#Definizione

Definizione 26 (game).

A *game* is a tuple $(\mathcal{S}, P, \sigma, t, f)_{a \in \mathcal{A}}$ where:

1. $\mathcal{S}$ is the set of states
2. P is the set of players, usually it's a numbered set from 1 to n
3. $\sigma : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is the successor function
4. $t : \mathcal{S} \rightarrow \{T, F\}$ is the *terminal test*, tells whether a game has ended or not
5. $f : \mathcal{S} \times P \rightarrow \mathcal{R}$ is a *reward function*, specified to a certain player

A solution for *a player* is the policy $\pi_P : \mathcal{S} \rightarrow \mathcal{A}$

Now we can distinguish two particular class of games:

Zero-Sum Games: Let $P = \{1, 2\}$. We say that the game is *zero sum* if P_1 wins *if and only if* P_2 loses. Intuitively, it's "*zero sum*" because at the terminal state the sum of the reward functions for each player is *zero*. With *zero sum games*, we can reduce the reward functions to only one, where a player wants to *maximize* the utility function, and the other wants to *minimize* the utility function.

General Games: On the other hand, if P is "*larger*" or agents have *independent utilities*, we have *general games* where multiple types of outcomes are possible, such as everyone having a win-win situation, or a restricted group of agents "*win*".

Search problems on *zero-sum games* are called "*adversarial search problems*".

Adversarial Search

X

Adversarial search problems. Intuitive representation of adversarial search problems: from $N = 1$ to $N = 2$. Adversarial game trees. Minimax search algorithm, techniques to deal with resource limits: alpha-beta pruning, depth-limited search and evaluation function heuristics. Uncertain games: motivation, expectimax algorithm, approaches in probabilistic modelling.

X

0. Voci correlate

- Definition of Search Problems
- Representation of Search States Space
- Definition of Games

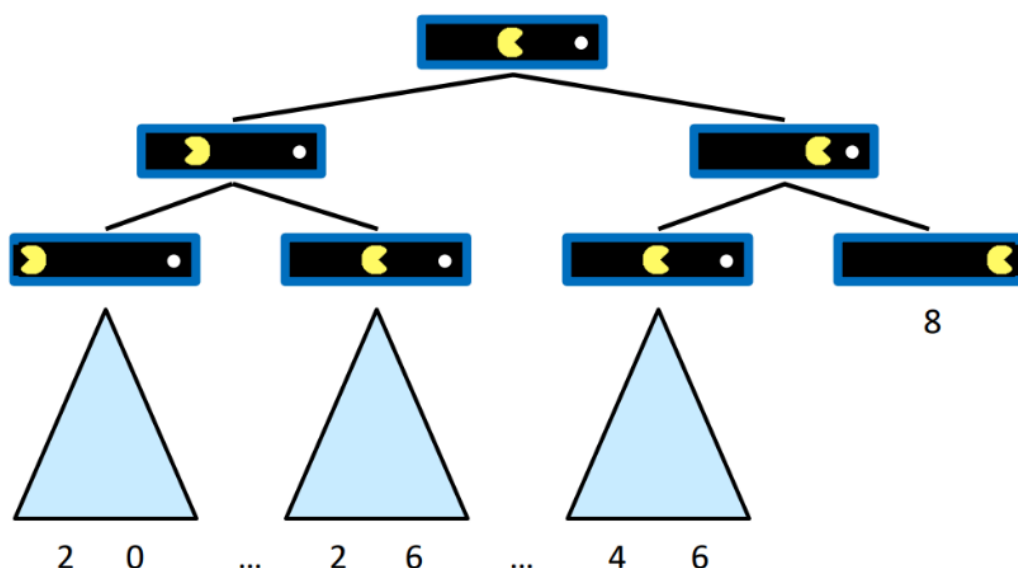
1. Adversarial Search Tree Representation

Q. How can we represent an *adversarial game*, in formal terms?

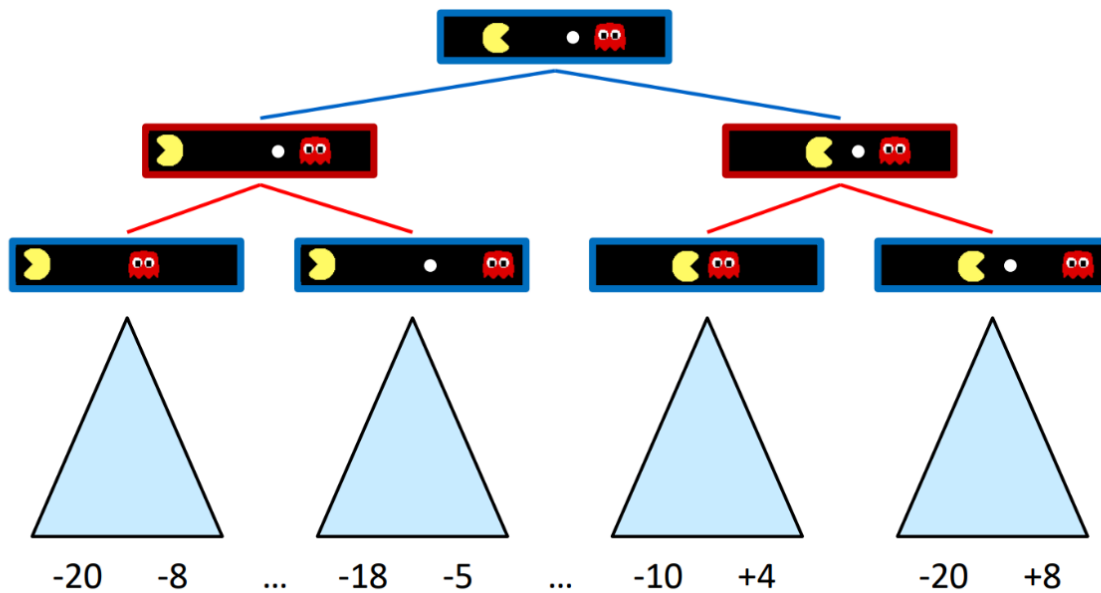
To give an answer, first we will think about the case where *only one player is involved*. Intuitively, in that case we just have a regular *search tree problem*, where we have a *search state space*, a *successor function* and a *reward function* to maximize, which is to be propagated layer by layer.

If the player is looking for an optimal solution, then they will have to search the *terminal nodes* and propagate their utility values to their ancestors, up to the root. In fact, we are just applying recursively the following two rules:

$$V(s) = \max_{s' \in \sigma(s, \bullet)} V(s'), \forall \{s \in \mathcal{S} : \neg t(s)\}$$
$$V(s) \text{ known}, \forall \{s \in \mathcal{S} : t(s)\}$$



So we can generalize this easily for the *adversarial case*; for each "layer" of the tree we have that it is one of the player's turns, and they can take actions on the current states. In our case, it is generally to be assumed that a player's turn is every two layers.



X

2. Minimax Algorithm

However, in this case we have to propagate the terminal values in a different way, in a such way that it accounts both the *player optimizing the value* and the *adversary minimizing the value* (as we're in a zero-sum game).

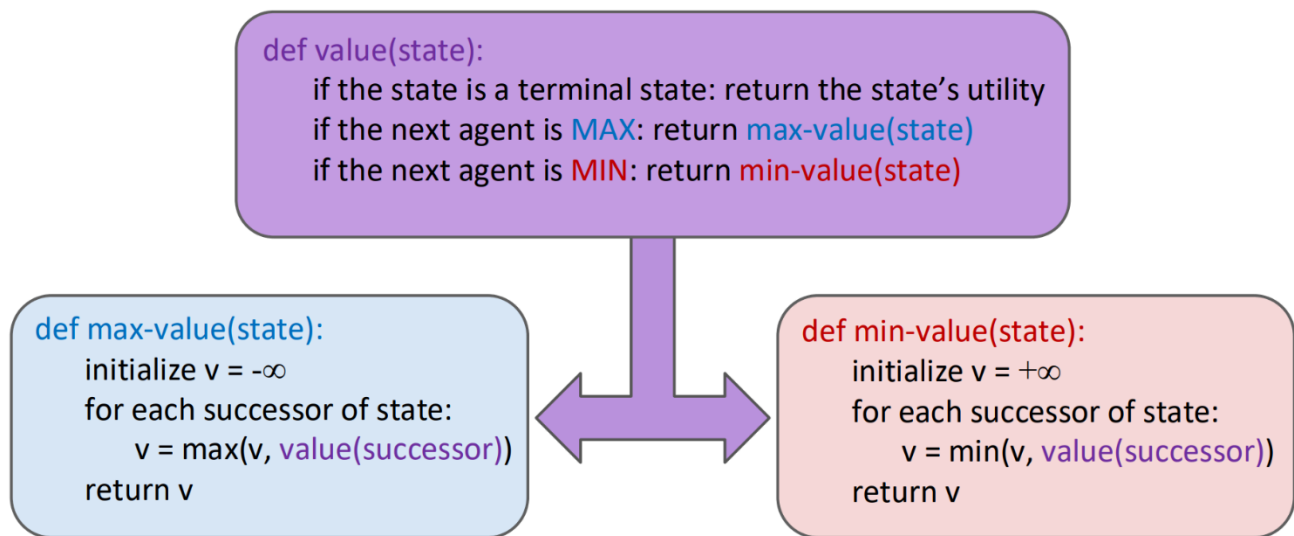
So we have three adversarial rules:

$$V(s) = \max_{s' \in \sigma(s, \bullet)} V(s'), \forall \{s \in \mathcal{S} : \neg t(s)\}, P = P_1$$

$$V(s) = \min_{s' \in \sigma(s, \bullet)} V(s'), \forall \{s \in \mathcal{S} : \neg t(s)\}, P = P_2$$

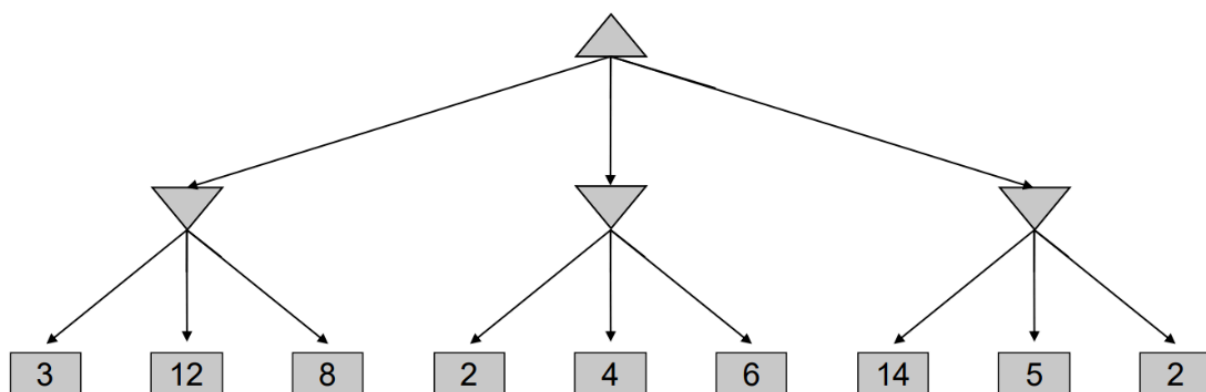
$$V(s) \text{ known}, \forall \{s \in \mathcal{S} : t(s)\}$$

The ideal for algorithm is *minimax* in this case, written as follows:



Notation. In the *tree graph* notation, we denote as follows:

- State nodes where the player is *maximizing the value* with an upper triangle symbol (Δ)
- State nodes where the player is *minimizing the value* with a lower triangle symbol (∇)



Properties. Minimax acts like an *exhaustive DFS*, meaning it has the following properties:

- *Time complexity*: $O(b^m)$
- *Space complexity*: $O(bm)$

2.1. Minimax with Resource Limitations

We can immediately see that the *original minimax algorithm* is a very expensive algorithm. For example, chess has a branching factor of 35 and the game can last around 100 steps! This means that we have around 35^{100} steps to do in order to compute the minimax of the values...

So we have to devise a *few tactics* to avoid exploring the whole tree, as most nodes are not *"needed to explore"*.

2.1.1. Alpha-Beta Pruning

Alpha-Beta Pruning. Conceptually, *alpha-beta pruning* is this: if you're trying to determine the value of a node n by looking at its successors, stop looking as soon as you know that n 's

value can at best equal the optimal value of n 's parent

In other words, the idea consists of pruning our tree by keeping track of α, β values which represent the "*lower and upper bounds*" of the value functions of the states we want to calculate. If, by exploring the terminal nodes, we find that they are "*outside*" of the range and has no influence on the α, β values then they can be *pruned* from the tree.

In other words, we're still running the minimax algorithm but we also keep track of minimum and maximum values and add them to the base case.

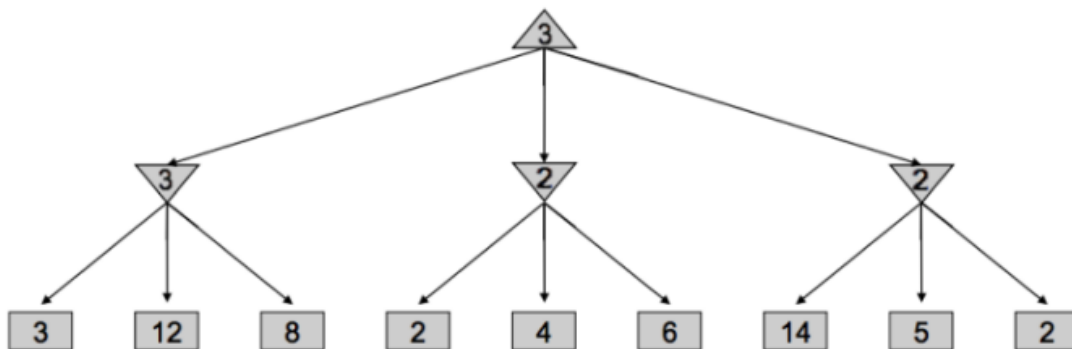
α : MAX's best option on path to root
 β : MIN's best option on path to root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \geq \beta$  return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \leq \alpha$  return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```

Esempio 27 (alpha-beta pruning example).

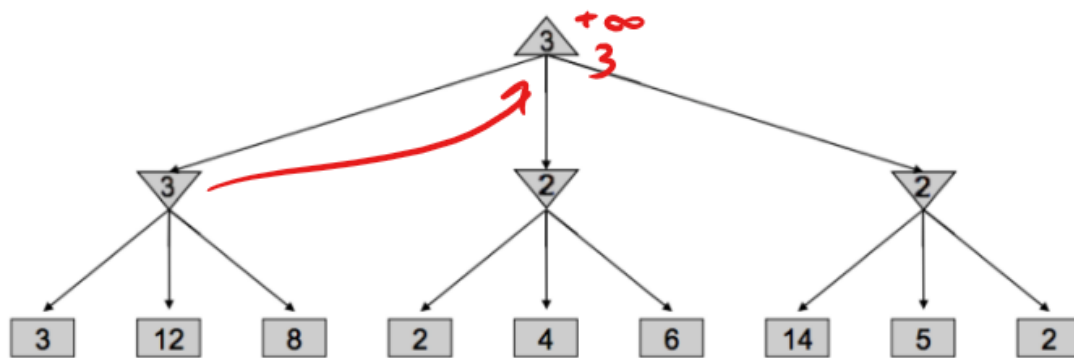
Given the game tree



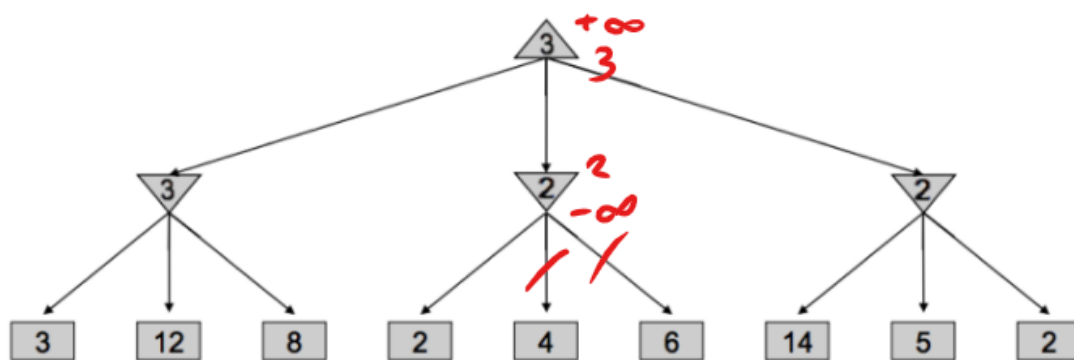
Assume we will visit nodes from left to right. We start by defining every nodes' lower and upper bound as $-\infty, +\infty$ (unless specified otherwise), and start visiting nodes to obtain more information about them.

Let us visit the left-hand side minimizer, and we visit its first leaf node: we have 3. Since it's a minimizer node, its *upper bound* α will be set to 3. Proceed to visit the other leaf nodes to see whether we can "improve" the bounds by finding lower values; turns out to be that there are no lower values.

So that node's value is exactly 3, which we will "propagate" by setting the root maximizer's lower bound to 3. Therefore, the root node's interval bound is $[3, +\infty]$.



Now let's visit the center minimizer: by visiting its first leaf node, we can infer that its bound range is $[-\infty, 2]$; however, the intersection between this and root's range is empty ($[3, +\infty] \cap [-\infty, 2] = \emptyset$), therefore we can "ignore" this minimizer by pruning the rest of the leaf nodes.



And then proceeding on the right side minimizer, we have to visit *all of its leaf nodes* as each of these values are greater than 3 (except the last one), therefore we do not prune any nodes.

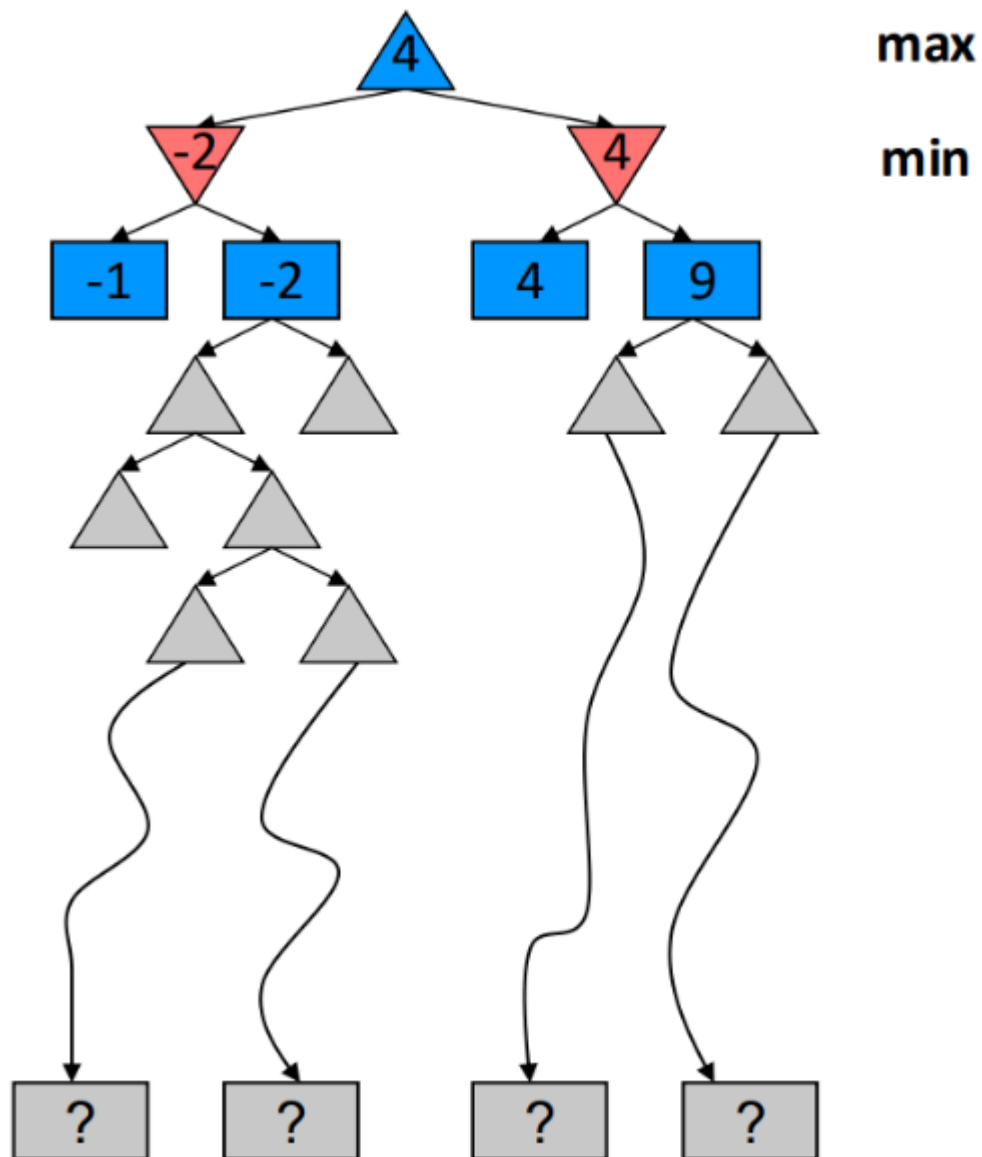
Properties. The alpha-beta transformation is a *sound transformation*, meaning that the pruning process has no effect on the calculated minimax value since it does not approximate the tree in any way. However, the value of *intermediate nodes* might be wrong, meaning that the transformation is not *equivalent*!

Remark. Ordering affects the performance of alpha-beta pruning, with *"perfect ordering"* reducing time complexity to $O(b^{m/2})$.

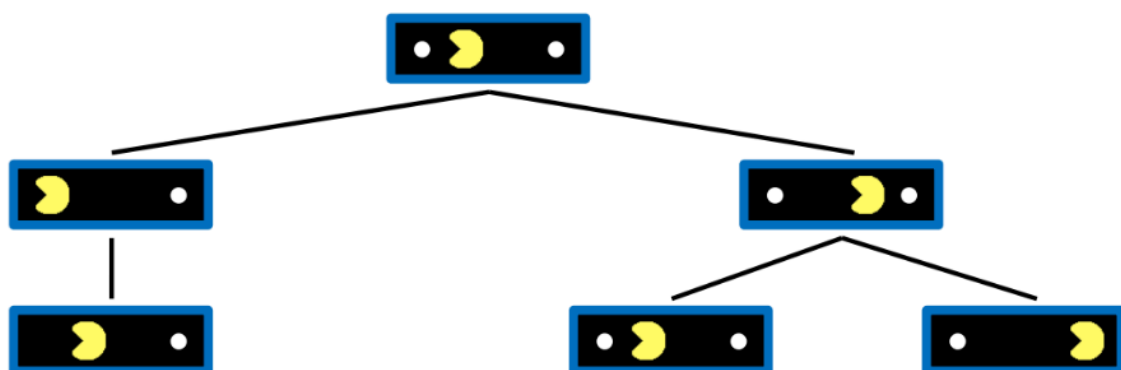
2.2.2. Depth-Limited MiniMax and Evaluation Functions

Another approach is to simply truncate the search tree.

Depth-Limited Search. We search *only* up to a limited depth in the tree and replace *terminal utilities* with an evaluation function for *non-terminal positions*. This causes a loss in optimality, as we will *"approximate"* the search tree.



Remark. While it *"usually works"*, there are still dangers to replanning agents. In fact, if the limit the tree search depth *too much* (e.g. $d = 2$), the agent will act in an unoptimal way. To give a clear idea, see the following counterexample:



As we can see, pacman avoids eating the dots as he only predicts up to two moves, so for him *"waiting"* seems as good as just eating.

Evaluation Functions. To define *heuristic evaluation functions* on the states, we typically return a linear combination of the features in the state. That is,

$$\text{eval}(s) = \langle \omega, f_s \rangle$$

The way we design the evaluation function is important, as it can determine the agent's behaviour.

Remark. We can combine *alpha-beta pruning* with the *function heuristic* technique, as it can provide a good *estimate* on how "*promising*" is a node, giving thus a "*good*" ordering of the nodes to visit first. Successively, we can apply *depth-limited search* for better performances.

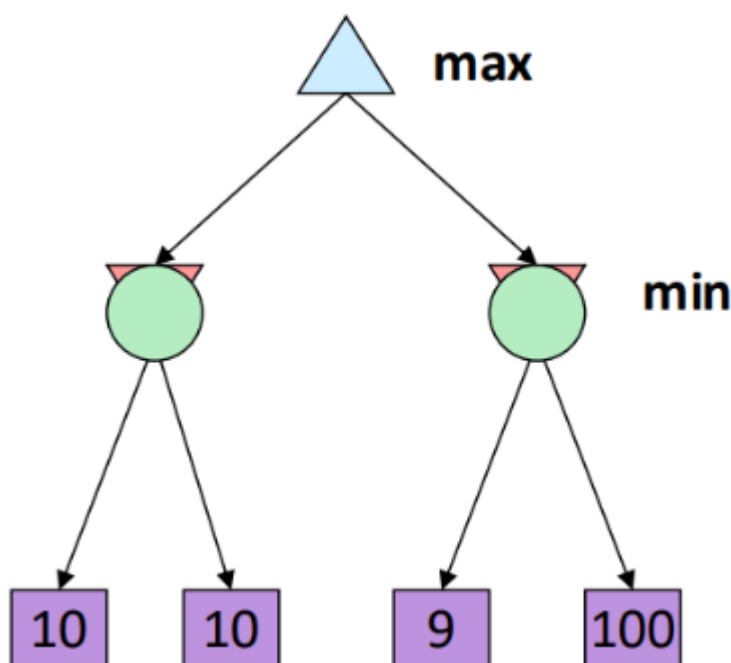
X

3. Expectimax Algorithm

Note how the *minimax algorithm* assumes that the opponent plays optimally.

Q. What if the environment is not fully observable? What if the opponent does not always play optimally? How much can I "*overshoot*" with minimax?

An example which underlines this aspect is as follows:



If we assume that the opponent plays optimally, we always end up with value 10. However, we can "*miss out*" on obtaining 100, with the slight "*risk*" of gaining 9.

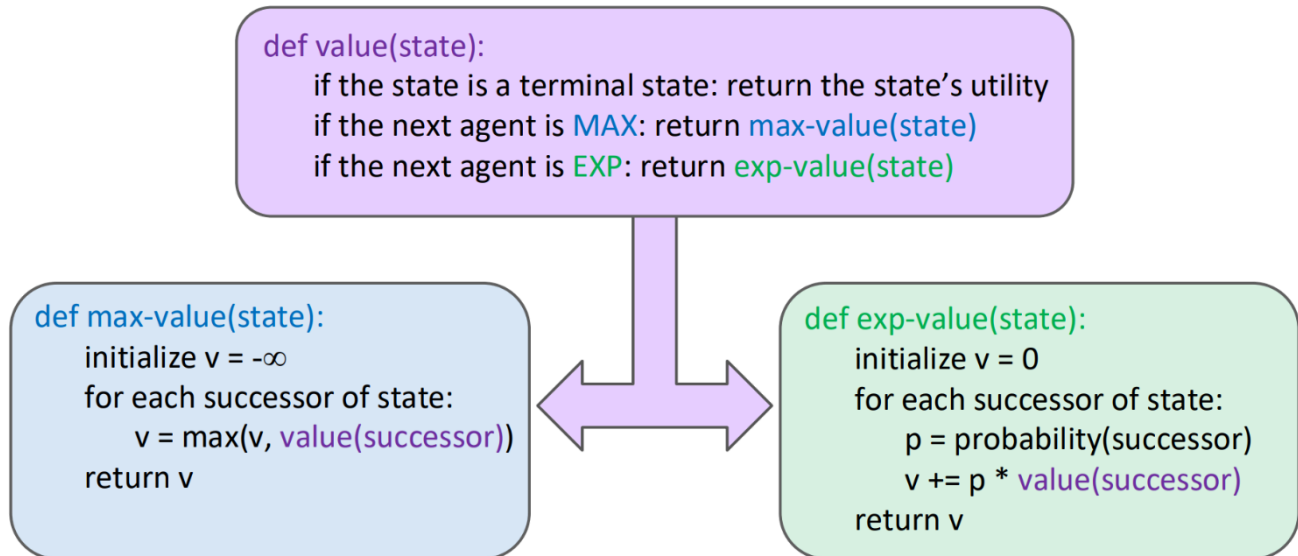
There are examples which lead to the adversary not playing optimally:

- *Explicit stochasticity*: there is an explicit stochastic factor involved in the game
- *Unpredictable opponents*

- *Faling actions*, in a "physical scenario"
- Philosophical considerations on "uncertainty"

To take into account the "random" factor, we will replace *minimum nodes* with "average nodes", and the values calculated for them is given by doing the weighted sums of their values according to their probabilities. We will denote those nodes with a *circle* ○

ALGORITHM. (*Expectimax*)



Q. Can we still apply *pruning techniques*?

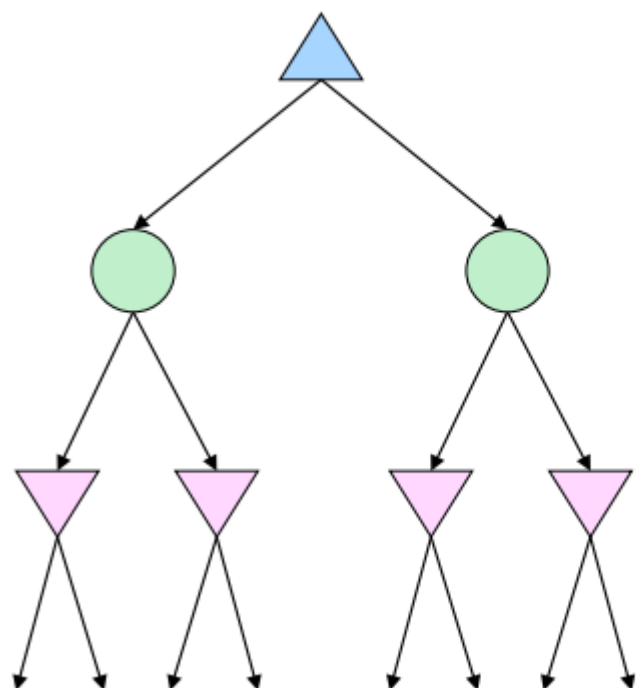
Not unless we have assumptions on the *range* of the values, otherwise we risk to "lose information".

To know which values p to use, we need to find or make assumptions about the opponent's probability distribution.

With this kind of modelling assumptions, we have two approaches:

- "*Dangerous optimism*": leans closer to expectimax, assuming chance when the world is adversarial
- "*Dangerous pessimism*": leans closer to minimax, assuming the worst when it's not likely

To conclude, we can generalize games with "*types of layers*"; there are certain adversarial games where there can be all of the three types of nodes combines (*maximizer, minimizer, chance*). An example of such game is *Backgammon*.



General Games

X

General games: generalization of minimax.

X

0. Voci correlate

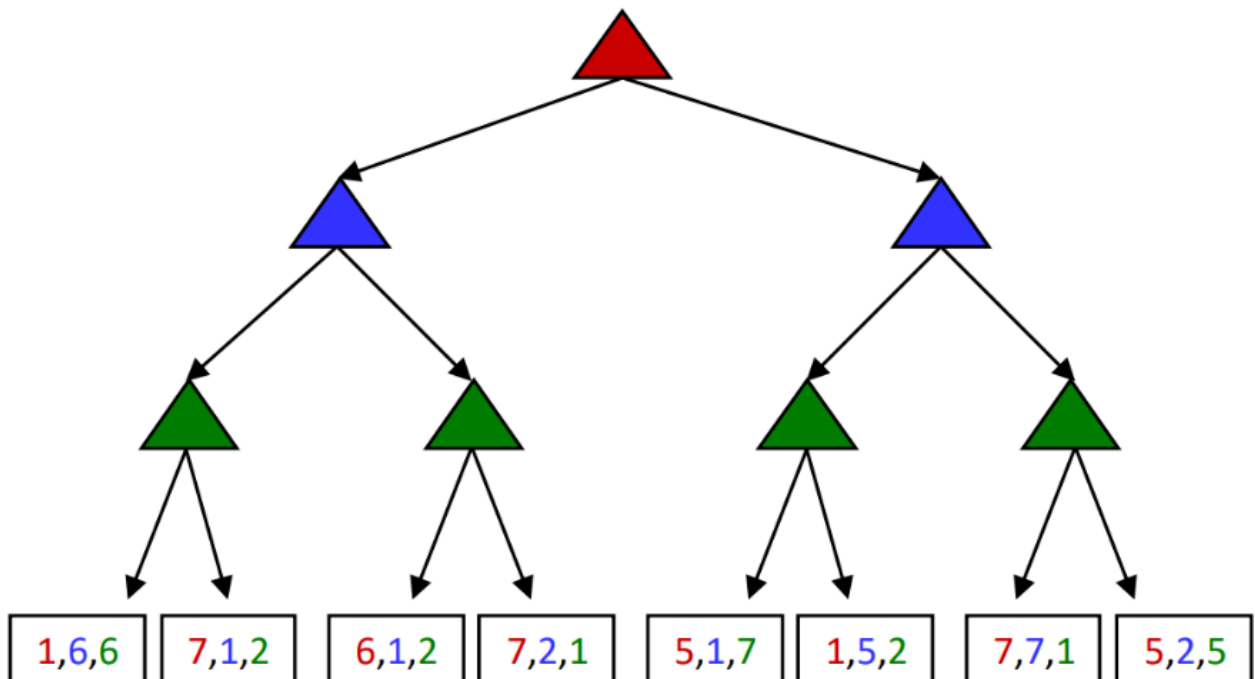
- Definition of Games
- Adversarial Search

1. General Games

Remark. What has been said about *adversarial search problems* can be generalized to *general games*, where:

- Terminal nodes have *utility tuples*
- Node values are also *utility tuples*
- Each player maximizes *its own component* and ignores the other player's utility values

So there can be win-win situations or partial win-win situations, allowing players to cooperate or compete in groups.



The red, green, and blue nodes correspond to three separate agents, who maximize the red, green, and blue utilities respectively out of the possible options in their respective layers. Working through this example ultimately yields the utility tuple (5, 2, 5) at the top of the tree.

General games with multi-agent utilities are a prime example of the rise of behavior through computation, as such setups invoke cooperation since the utility selected at the root of the tree tends to yield a reasonable utility for all participating agents.

Remark. Unfortunately, pruning techniques cannot be applied in this situation.

Monte Carlo Tree Search

X

Monte Carlo tree search: idea of the algorithm, "rollout" and "selective search". Version 0, 0.9, 1.0, UCB formula and version 2.0 (UCT algorithm).

X

0. Voci correlate

- Definition of Games
- Adversarial Search

1. Idea of Monte Carlo Tree Search

Unfortunately, using *resource limiting techniques* on minimax is not sufficient for certain problems. For instance, Go has a branching factor of 300, making it hopeless to use alpha-beta pruning or similar techniques.

IDEA. *Monte Carlo Tree Search* is an algorithm which combines two ideas:

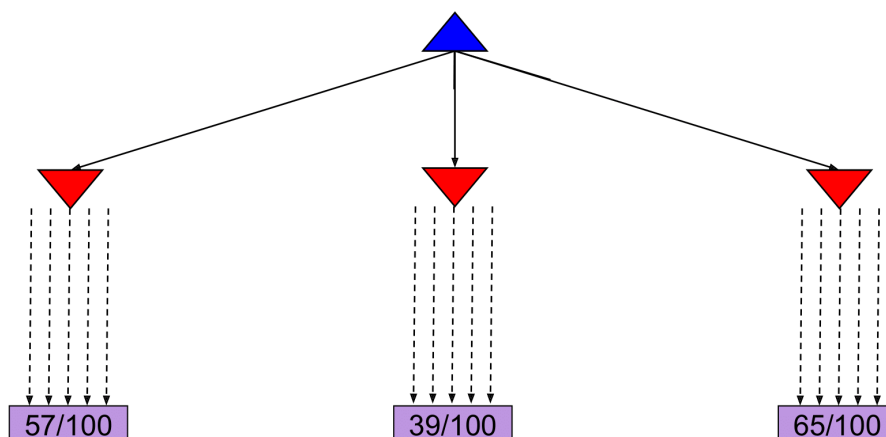
- Simulating with a *"fast policy"* (*rollout*)
- *Selective search*, explore parts of the tree that will improve the decision at root.

It's as if I *"watched games"* and *"gained experience"* by observing the games and calculating statistics.

X

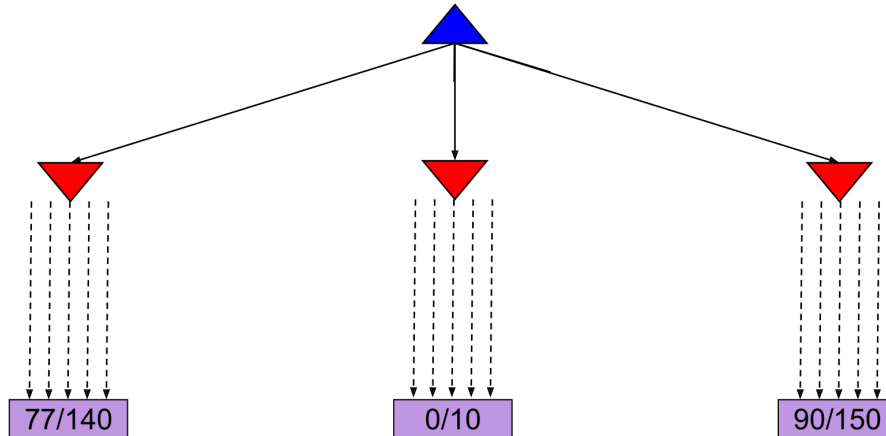
2. Implementations of MCTS

Version 0. Do N rollouts from each child of the root, record the fractions of wins $U(n)/N(n) = U(n)/N$. Choose the action which leads to the node with the highest ratio of wins.



Note that if values are similar or they are 0, we cannot really do anything...

Version 0.9. In addition, discard nodes *with value 0*, allocate more rollouts to *"more promising"* nodes.



However, we still risk having *bigger variances* between uncertain nodes; some actions yield similar percentages of wins, but one of them has used far fewer simulations to estimate that percentage. Hence we want to allocate more rollouts to the actions with higher variance.

Version 1.0. Also allocate more rollouts to uncertain nodes, i.e. with a potential *"higher variance"*.

We can calculate the trade off between allocating *"promising"* and *"uncertain"* nodes with the following formula:

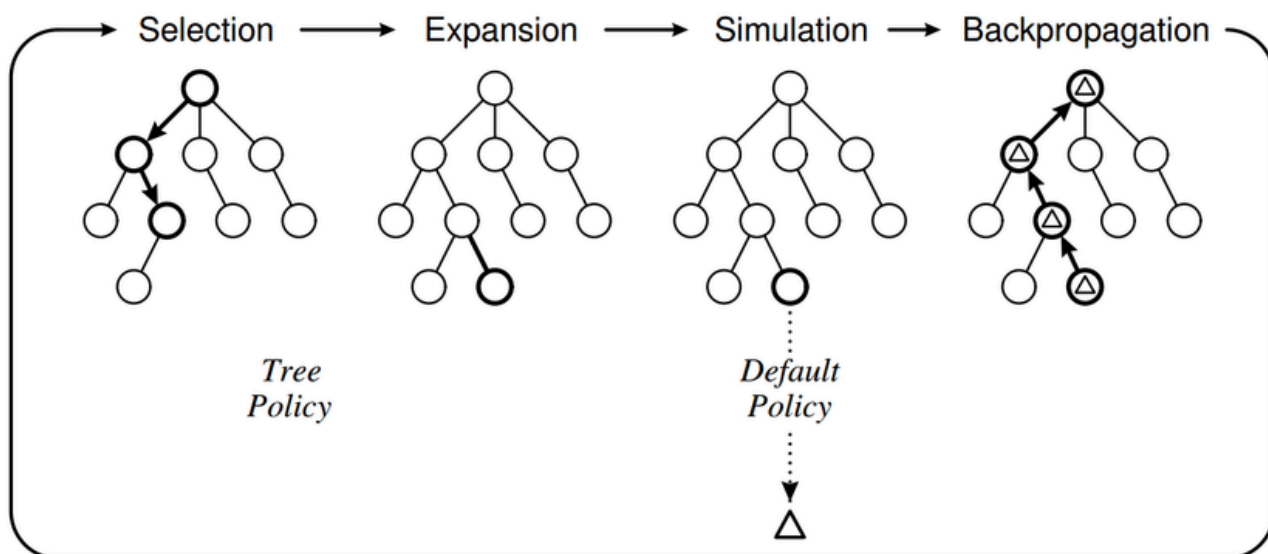
$$UCB(n) = \frac{U(n)}{N(n)} + C \sqrt{\frac{\log(N(\text{PARENT}(n)))}{N(n)}}$$

Where C is a hyperparameter, usually $C = \sqrt{2}$.

Version 2.0. (UCT) Use the UCB criterion to choose which node to simulate, that is to repeat the following steps:

- Use the $UCB(\bullet)$ function to move down the layers of the tree until an unexpanded leaf node is reached
- Add the new child to the leaf and run a rollout from that child, determine the number of wins
- Propagate the wins to its ancestors until the root

It can be proven that for $N \rightarrow +\infty$ the *UCT algorithm* selects the *minimax move*. The intuitive idea behind this is that the vast majority of rollouts are concentrated in the best children on the long run, so the weighted average becomes *maximum/minimums*.



Markov Decision Processes

Markov Decision Processes

X

Introduction to Markov Decision Processes. Type of search problem: sequential decision problems. Definition of a Markov Decision Process. Equivalence between Markov Decision Processes and Game Tree Graphs (MDP Search Trees). Discounts: utilities of sequences, finite and infinite horizon.

X

0. Voci correlate

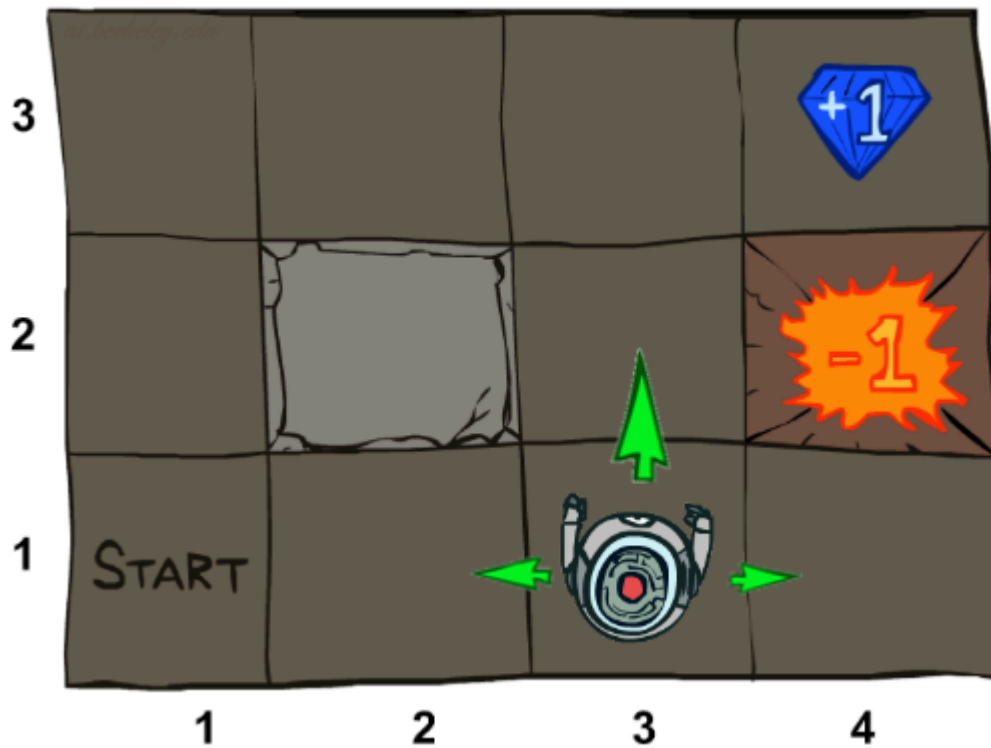
- Processi Decisionali di Markov
- Adversarial Search

1. Definition of Markov Decision Processes

Now let us define a class of *non-deterministic search problems*, which is important as assuming that every action an agent takes has "*100% success rate*" is extremely optimistic for real-world applications. In particular, *sequential decision problems* will treat problems where that the agent's utility depends on a *sequence of decisions*, and for each state we know *what actions* an agent can take, but we won't know which one will be taken.

To take this sort of uncertainty into account, we will need to compute "*policies*" which consider all the different outcomes that might occur in response to a certain action taken in a certain state.

Example. Grid World, a maze-like problem where the agent lives in a 2D grid and there are walls blocking the agent. They move "*randomly*", with 80% chance of taking the direction he's facing and 10% on the "left" side and the other 10% on the "right" side. There are some tiles which can cause the agent to *gain* (or lose) rewards. The agent's goal will be to determine a policy which creates a sequence of actions that will *maximize* the sum of rewards.



#Definizione

Definizione 28 (Markov Decision Process).

A *Markov Decision Process* is defined with the following construction:

- A set of states $\mathcal{S}$
- A set of "global" actions $\mathcal{A}$, we can denote $A(s)$ as the set of legal actions for the state s
- A transition function (*model/dynamics*) $T(s, a, s')$ which is the probability of the agent leading to the state s' if he takes the action a at state s (so $p\{S_{t+1} = s' \mid S_t = s, A_t = a\}$)
- A *reward function* $R(s, a, s')$ or sometimes just $R(s)$ or $R(s')$
- A *start state* $s_0 \in \mathcal{S}$ and *optionally* (!) a set of *goal states*

Given a MDP, a *policy* is a tool for the agents to "*plan out*" their actions, in other words it's a stochastic or deterministic mapping $\pi : \mathcal{S} \rightarrow \mathcal{A}$. An *optimal policy* is the one policy that "*maximizes the reward*" if followed (we will see the meaning behind this on a deeper level) and it will be denoted with π^* .

Note how *expectimax* cannot solve this problem, as it only plans the action for a *single state only*!

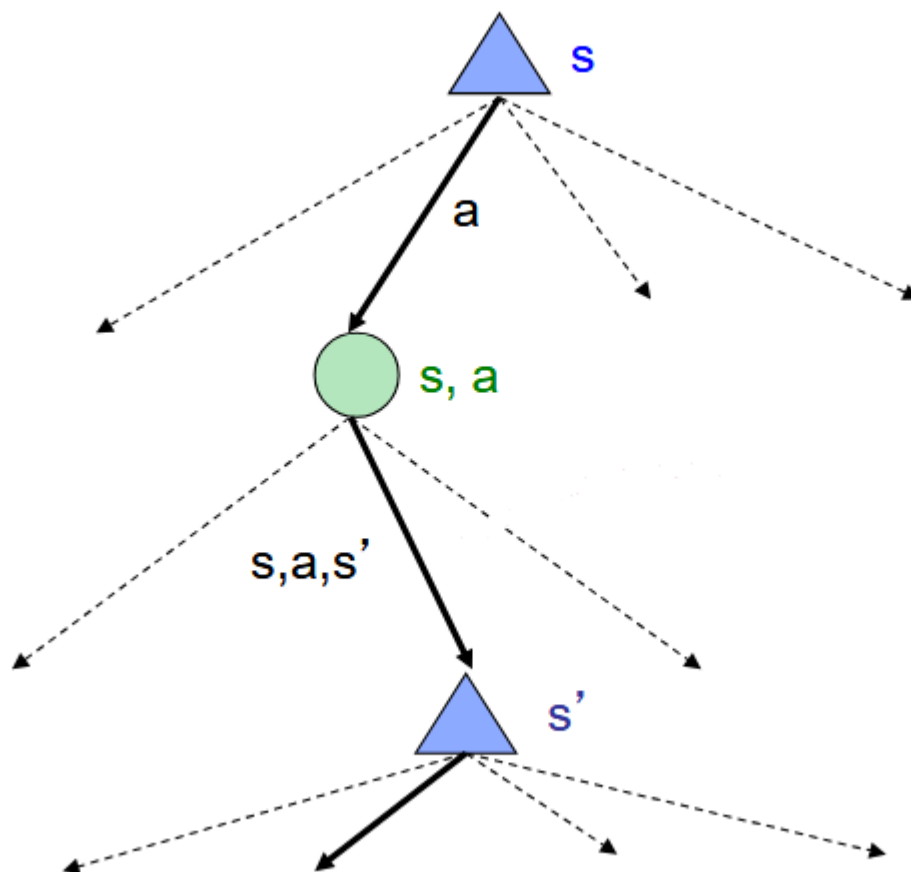
2. Properties of MDPs

Property. (Markovian) The term "*Markov*" is due to the "*markovian*" property of such set, in fact the *transition function* is designed in such a way that it ignores the "*history*" of an agent when taking an action. In other words, the transition function is dependent only on the current state and action:

$$p\{S_{t+1} = s' \mid S_t = s, A_t = a, S_{t-1} = \dots, A_{t-1} = \dots, S_0 = s_0\} = p\{S_{t+1} = s' \mid S_t = s, A_t = a\}$$

Property. (MDP Search Trees) Note that *MDP states* can be unraveled into search trees "*similar to*" expectimax search trees. Infact, if we consider the triple state-action-successor (s, a, s') , we have that:

- s is a state represented as a *maximizer*
- s, a descends from s and is represented as a *chance node* ("*action state*"); this models the uncertainty of the environment
- s' is another *maximizer* which descends from s, a



3. Discounting

Remark. Note how that up until now, the agents prefer only to *maximize* the *sum of rewards* in a MDP problem. However, it is preferable to have a policy which maximizes the reward either in the "short term" or in the "long term".

Discount Factors Generally, it's reasonable to prefer reward *now* to *rewards later*. For example, an agent should produce the rewards $[1, 0, 0]$ rather than $[0, 0, 1]$.

To solve this, we will apply an *exponential decay* to the rewards with the term $\gamma \in [0, 1]$. This technique is called "*discounting*" and γ is a *discount factor*.

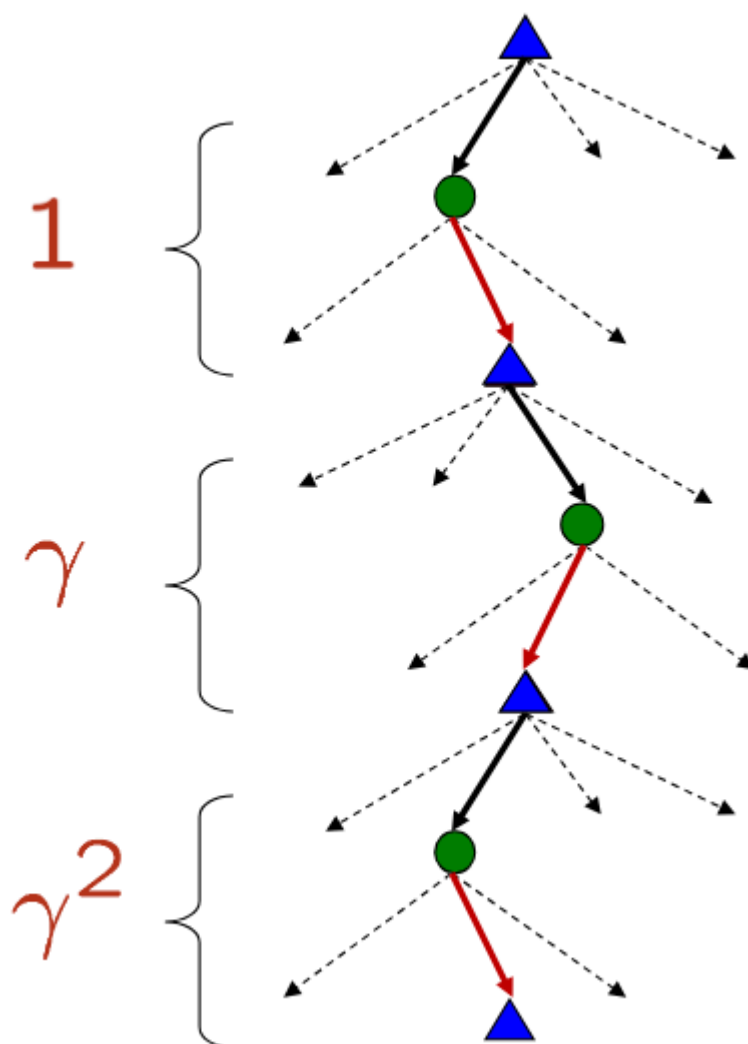
We have that, given a history $\mathcal{D} = \{s_0, a_0, s_1, a_1, \dots, s_n\}$, its *discounted reward* is

$$U(\mathcal{D}) = \sum_{i=0}^{n-1} \gamma^i R(s_i, a_i, s_{i+1})$$

Intuitively, for $\gamma = 0$, there is no discounting factor and the agent will have a "*deep*" view on the model; whereas, for $\gamma = 1$ the agent will have a "*miopic*" view of the MDP and will tend to greedily choose the best action.

If $\gamma = 1$, we have an *additive utility*. Otherwise, it's a discounted utility.

In *MDP search trees*, we have that "*shallow rewards*" are more preferable to "*deep rewards*".



Remark. If the horizon is "*infinite*", i.e. $n = +\infty$ meaning that the game "*lasts forever*", then *discounting* can be useful for $0 < \gamma < 1$ as the sum of rewards converges. Let us prove this.

#Lemma

Lemma 29 (discounting with infinite horizon).

Given an agent in an *MDP environment* and with an "*infinite*" history of rewards

$$\mathcal{R} = \{R_0, \dots, R_n, \dots\}$$

such that it's bounded, i.e. $\exists R_{\max}$ such that $R_{\max} \geq R_i, \forall i$.

Then for $0 < \gamma < 1$ the *reward sum* converges:

$$\sum_{i=0}^{+\infty} \gamma^i R_i < +\infty$$

Another solution to the problem of "infinite horizon" is to use *absorbing states*, guaranteeing that for each policy a terminal state will be eventually reached.

#Dimostrazione

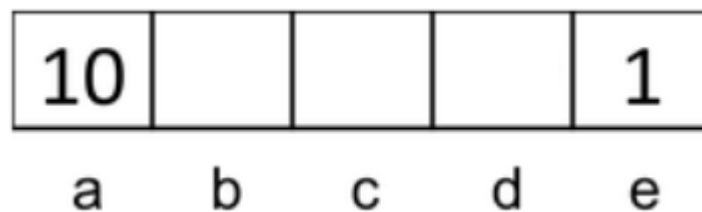
PROOF of

Apply an upper bound and calculate the geometric series:

$$\sum_i \gamma^i R_i \leq \sum_i \gamma^i R_{\max} = \frac{R_{\max}}{1 - \gamma}$$

Concluding. ■

EXERCISE. Given the following world



Where each tile is a state, and *a* has reward 10, *e* has reward 1. Suppose the actions are either East or West or Exit (only for *a* and *e*), and that the transitions are *deterministic*.

1. Which is the optimal policy for $\gamma = 1$?
2. Which is the optimal policy for $\gamma = 0.1$?
3. For $s = d$, determine the value γ such that both goal states have the same discounted reward value.

Solving MDPs

X

Solving Markov Decision Process. Definition of value function, proposition: Bellman equation. Definition of optimal policy. Definition of action-value function (Q-value), Bellman equation. Methods to calculate value or action-value functions: value iteration, policy iteration.

X

0. Voci correlate

- Solving Markov Decision Processes
- Processi Decisionali di Markov
- Policy, Value Iteration

1. Value and Action-Value Functions

GOAL. Determine a way to find an optimal policy π^* for a *MDP*.

Before determining a way to find said policy, we should introduce two mathematical quantities first, which measure performance of an agent's policy.

In particular we will see 3 functions.

#Definizione

Definizione 30 (optimal value).

The *value* of a state s is defined as the *expected value* of the sum of discounted rewards an optimal agent starting from state s and acting according to policy π . That is,

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k \geq 0} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

#Definizione

Definizione 31 (optimal action-value function).

The *action-value* of a pair action-state (a, s) is similarly defined as the *optimal value* for a state s , except that the action a is also involved in the calculation of the expectation.

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k \geq 0} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

#Definizione

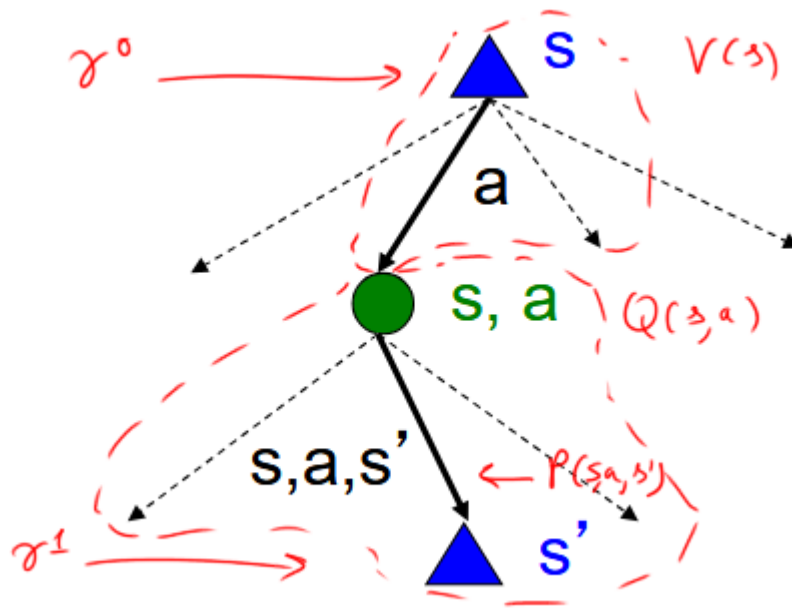
Definizione 32 (optimal policy).

A policy π^* is said to be *optimal* if it satisfies both of the *optimality equations*:

$$V^{\pi^*}(s) = \max_{\pi} V^{\pi}(s), Q^{\pi^*}(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

The fundamental operation to calculate V^{π}, Q^{π} is to *compute the value* of a state. In terms of *MDP trees*, we are calculating the *expectimax* value of a state, considering the *average sum of discounted rewards*.

$V^*(s)$ corresponds to calculating the *expectimax* of the maximizer node (s), $Q^*(a, s)$ corresponds to calculating the *expectimax* of the chance node (s, a).



By the calculations effectively, we obtain the *Bellman optimality equations*, which characterise the *optimal value functions* through the following definitions.

#Teorema

Teorema 33 (Bellman Optimality Equations).

The definition of optimal *value, action-value* functions can be defined as follows:

$$(B_1) \quad V^*(s) = \max_a Q^*(s, a)$$

$$(B_2) \quad Q^*(s, a) = \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Note how substituting (B_2) into (B_1) yields the following Bellman equation:

$$(B_3) \quad V^*(s) = \max_a \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Similarly, by substituting (B_1) into (B_2) we obtain this:

$$(B_4) \quad Q^*(s, a) = \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma \max_a Q^*(s', a)]$$

Remark. Note how the *Bellman Optimality Equations* relies on itself to define itself. This fact will be useful for applying *dynamic programming techniques* to solve MDPs!

We can generalize the *Bellman equations* on any policy π by "*replacing*" the choice of maximal a with $\pi(s)$.

- $V^\pi(s) = \sum_{s'} P(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$
- $Q^\pi(s, a) = \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma Q^\pi(s', \pi(s'))]$

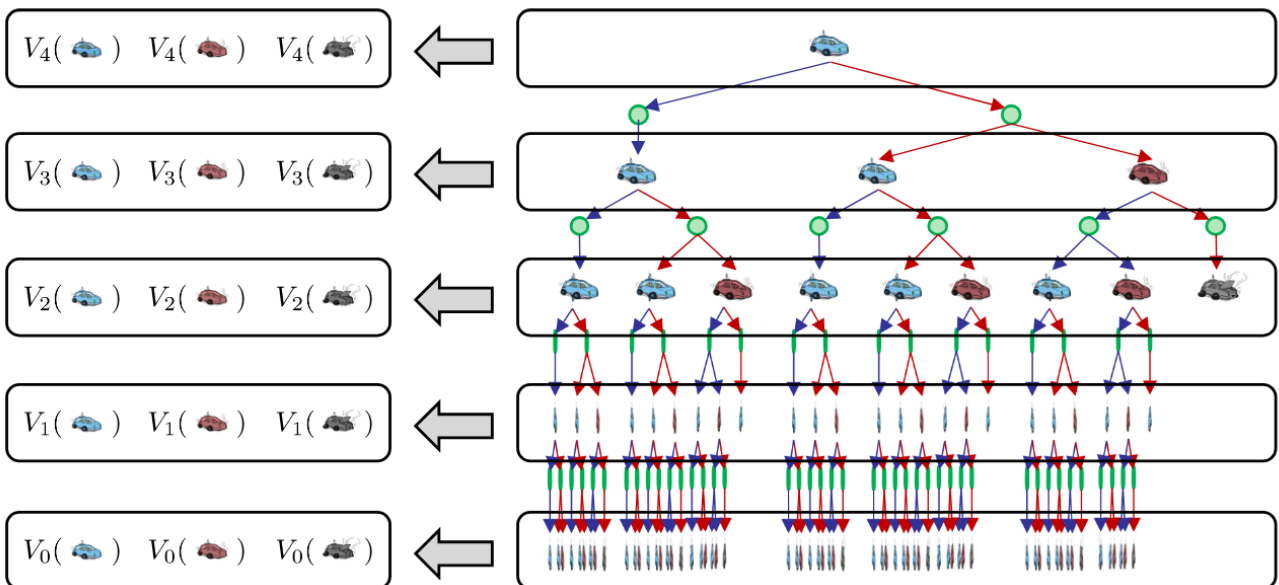
Note how for fixed π the V^π, Q^π values can be calculated with direct methods. However, such thing is not possible for the optimal values, as the equations lose linearity with the max operator.

X

2. Value Iteration

Assuming that we *know* the MDP model, we can calculate the V/Q functions in an accurate way by applying the Bellman equations recursively. The only problem is that the tree *grows infinitely* in most cases, therefore it will necessary to impose a *depth limit* on expectimax.

We will denote the utilities U_k (where U is either V or Q) as the k depth-limited calculations of the expectimax.



Value Iteration. Let $V_0(s) = 0$, as *no time steps left* mean that the expected reward is zero. Given $V_k(s)$, explore the next layer of the expectimax. In particular the update is given as follows:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

The idea is to let $k \rightarrow +\infty$ and eventually the "*approximated*" value function converges towards the optimal value function.

The complexity of each iteration is $O(S^2A)$, as:

- We need to calculate $V(s)$ for each state
- We must iterate over $\mathcal{A}, \mathcal{S}$ for each calculation of $V(s)$

#Teorema

Teorema 34 (convergence of value iteration).

The *value iteration* algorithm converges towards the *optimal value function*. That is,

$$\forall s, V_n(s) \longrightarrow V^*(s)$$

#Dimostrazione

PROOF of ^a29985 (*Sketch*)

We will use the notion of *contraction* and *fixed points* to prove the convergence. If we set the *bellman update step* (^2738a3) as an *operator* that we call B , we have that

$$V_k = B(V_{k-1})$$

The goal will be to find the fixed point V^* , if it exists.

To measure the *utility functions*, we cannot just calculate their absolute value as they are actually *vector functions*. So to measure them we will take the infinity-norm

$$\|V_k\|_\infty := \max_s |V(s)|$$

Recall that for the *fixed point method* to succeed, it must hold that B is effectively a contraction.

Please believe me when I say that the Bellman operator is a contraction under the infinity-norm, that is there exists a $\gamma \in (0, 1)$ such that

$$\|BV_k - BV'_k\|_\infty \leq \gamma \|V_k - V'_k\|_\infty$$

Which means that the fixed point method will converge to its fixed point (which exists, as we made a leap in the faith by believing that the Bellman operator is a contraction), by recursion:

$$\|BV_k - BV^*\|_\infty \leq \gamma \|V_{k-1} - V^*\|_\infty$$

Applying this step k times we obtain

$$\|BV_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$$

As V_0 is defined as the null vector by definition, we have that

$$0 \leq \|V_{k-1} - V^*\|_\infty \leq \gamma^k \|V^*\|_\infty$$

So by the squeeze theorem (or the "*two police agents theorem*") the central argument converges to 0, meaning

$$\lim_k \|V_k - V^*\|_\infty = 0$$

Meaning that the iterative method converges to the optimal solution in infinity-norm:

$$V_k \xrightarrow{\infty\text{-norm}} V^*$$

Concluding. ■

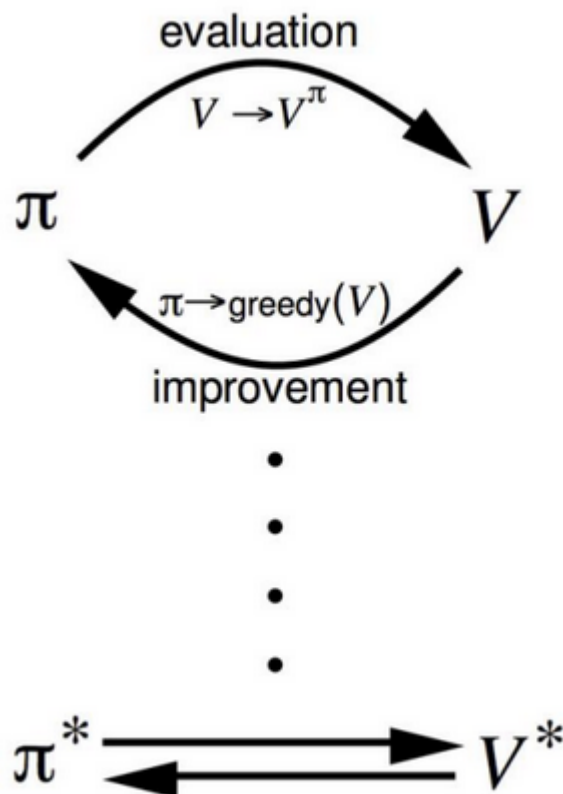
X

3. Policy Iteration

The problem with *value iteration* is that it is *computationally slow* and usually the changes achieved are "*small*".

We will use a *policy-based method*, which follows the following update paradigm:

- *Policy Evaluation*: Compute the utility function, either V or Q according to a policy π
 - We can either use *value iteration* for fixed π or "*remove*" the maxes, turning the Bellman equations into linear systems which can be easily solved
- *Policy Improvement*: Compute a "*better policy*" π' which is "*better*" than the old policy π (usually based on the Bellman equations)



Policy iteration is an example of a method that uses this layout.

Policy Iteration. Initiate π_0 arbitrarily or randomly and $Q^{\pi_0}(s, a)$ for all s, a . Iterate as follows:

- *Evaluate* $Q_i^{\pi_i}$ with the Bellman equations

$$Q_i^{\pi_i}(s, a) = \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma Q_i^{\pi_i}(s', \pi(s))]$$

- *Improve* π_i with π_{i+1} as follows:

$$\pi_{i+1}(s) = \arg \max_a Q_i^{\pi_i}(s, a)$$

Note that if we combine the *evaluation* and *improvement* steps, we have something called *policy extraction* as we are "*extracting*" the policy from the Q values. The intuition behind this idea is to "*think*" that $Q_i^{\pi_i}$ has already converged towards the optimal Q-values Q^* .

The *time efficiency* of *policy iteration* is $O(S^2)$ per iteration, which makes it faster than *value iteration*.

nothing serious

Topics on Knowledge Representation and Planning

X

Topics in knowledge representation and planning: important terminologies in knowledge representation (ontology, ontological engineering, objects and belief), approaches to planning (STRIPS heuristic, Sussman anomaly, Partial-Order-Planners)

X

1. Knowledge Representation

Warning

N.B. Some (most) parts of these notes are directly taken from other students' notes (Ludovico Urbani) or are revised with ChatGPT. NGL, this part is probably useless as the lecture on this part has been extremely rushed (explained all of this just in an hour...), so I didn't take this seriously

Knowledge representation is a cornerstone of Artificial Intelligence (AI), allowing systems to model and reason about the world. Effective representation bridges abstract concepts with computational frameworks, enabling intelligent agents to make informed decisions.

Knowledge representation provides AI agents with tools to reason, adapt, and make decisions in complex environments. By leveraging ontologies, categories, and reasoning systems, agents can process and integrate information effectively, ensuring intelligent behavior in dynamic contexts.

We will see some *important* terminologies in the context of Knowledge Representation, as follows:

Ontology. Ontology is, to simply put, a map of meanings independent from the syntax. From this we have *upper ontology*, which is a general framework of concepts to make simplifying assumptions.

There are two types of ontologies:

- *General-purpose*: provides a general conceptual structure for simplifying domain-specific representation. So there are no issues in *representation*, however it is impractical.
- *Special-purpose*: intuitively, ontologies applied to a specific *domain*

An example of ontology is *Google Knowledge Graph*, which is general-purpose.

Categories and Objects. Just like in Objected-Oriented-Programming, we let objects belong in *categories*, which may inherit some properties.

Physical Composition. At the beginning, to recognize objects we used the concept of *composition*. It is a particular type of category which divide objects. Intuitively, we can think of the human body as a *physical composition of more parts* ("objects")

Measurement. We can assign values to objects for measurement, as AI needs *quantification*

Events and Actions. To reason about *events* and *time*, AI systems need a way to do that. Simply, they use calculus to do that as we can represent events as *fluents*, i.e., properties which change over time.

Modal Logics. Agents rely on *mental models* to be able to express beliefs

Note how that there's no *single* way to represent things!

X

2. Planning

A *planning problem* asks for an *action sequence* that, when executed from an *initial state*, reaches a state that satisfies a *goal description*.

So, given actions (operators), an initial world state, and a goal, we want to compute a *plan* (sequence of action instances).

Planning is often described as *more powerful than generic problem solving* because it uses richer representations:

- States, goals, actions are expressed as *sets of logical sentences* (often first-order logic style).
- This can allow *subgoals* to be reasoned about (sometimes independently), which can reduce complexity.
- Many planners search in *plan space* (reasoning about which actions matter) rather than blindly exploring the full *state space*.

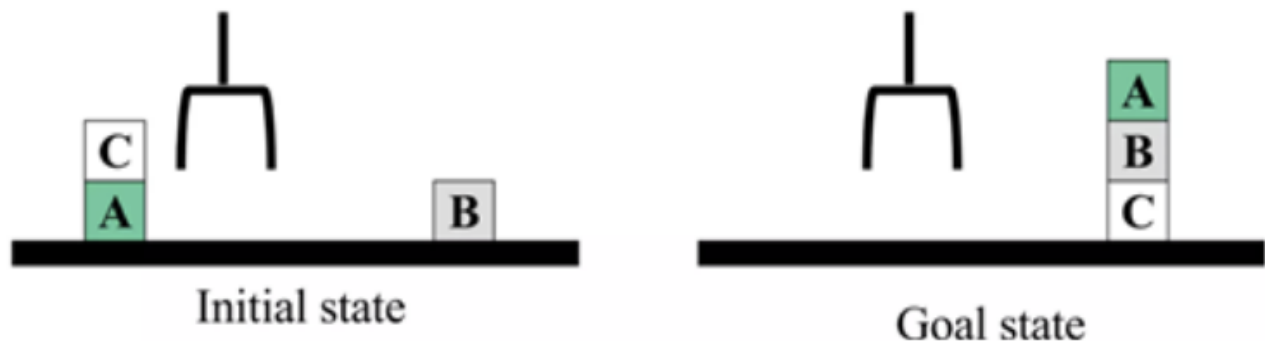
Classical Planning. Typically, we assume the following simplifications:

- *Atomic time*: actions are indivisible.
- *No concurrency*: actions aren't executed in parallel (though a plan may leave some steps unordered in partial-order planning).
- *Deterministic actions*: effects are fully determined (no uncertainty).
- *Agent is the only cause of change* in the world.
- *Complete knowledge*: the agent fully knows the current state.
- *Closed World Assumption*: anything not stated as true is treated as false.

STRIPS. One of the first classic planning frameworks is STRIPS (circa 1970), that represents:

- *States* as a conjunction/set of *ground literals* (facts like `at(Home)`, `handempty`, etc.).
- *Goals* as a conjunction of literals (may include variables; typically existentially quantified)
- Actions (operators) with:
 - *Preconditions*: what must be true before applying the action
 - *Effects*: what becomes true/false after applying the action
 Often implemented via *Add list* (facts to add) and *Delete list* (facts to remove).

Sussman Anomaly. A key limitation of simple STRIPS-style goal stacking is working with *subgoals*, and the *Sussman Anomaly* is an example of this problem.



State-Space Planning. We have two kinds of *state-space planning*, which work through the situations to get to the goal

- *Progression (forward) planning*: search forward from the initial state toward the goal.
- *Regression (backward) planning*: search backward from the goal, asking “what states could have led here?”
 - *Intuition*: can reduce branching because it focuses on actions relevant to the goal, but it relies on being able to reason backwards using operator descriptions.

POP Planning. There is another particular type of *planners*, called *partial-order* planners. Instead of searching over states, they search over *partial plans* and refine them.

This page intentionally left blank.

everybody gangsta till
the chest starts walking

